

应用于 MNIST 的 MLP 与自动微分引擎

项目报告

AUTEURS

Jianye Shi

jianye.shi@ens.uvsq.fr

Hao Qian

hao.qian@ens.uvsq.fr

Xiang Bian

xiang.bian@ens.uvsq.fr

Abdenmour Boulmis

abdenmour.boulmis@ens.uvsq.fr

ENCADREMENT

Aurélien Delval

Enseignant-chercheur

Table des matières

1	引言	1
1.1	项目背景与研究对象	1
1.2	前期基础与本阶段扩展	1
1.3	报告范围与后文内容	1
2	GEMM 矩阵乘法算子的优化	1
2.1	实验环境	2
2.2	线程预算与实验协议	2
2.3	OpenMP 分块与打包技术	2
2.4	GotoBLAS 架构	2
2.4.1	三层分块模型	2
2.5	GotoBLAS 调优与多线程并行策略	3
2.5.1	微内核设计	3
2.5.2	结果摘要	4
2.5.3	最终 AVX2 配置与 AVX-512 简介	4
2.5.4	GotoBLAS 多线程并行策略与参数调优	4
2.5.5	线程感知块参数调优	4
2.5.6	强扩展性实验 (Strong Scaling)	4
2.6	Unrolled+Prefetch 实现思路	5
3	损失函数、优化器与训练循环	5
4	卷积算子的实现和优化	6
4.1	卷积层的初始实现方式	6
4.2	卷积层并行化与切分策略	7
4.3	实现细节与工程取舍	7
4.4	对照与观测指标	7
5	CNN 架构的实现	7
5.1	当前 CNN 结构与数据流	8
5.1.1	整体结构	8
5.1.2	前向数据流、存储表示与统一接口	8
5.2	运行验证	9
5.3	ResNet 结构扩展	9
5.3.1	结构与下采样策略	9
5.3.2	BasicBlock	9
6	基于 Optuna 的超参数自动搜索	10
7	Tiny-ImageNet 数据集迁移与输入管线	10
7.1	迁移动机与数据特征	10
7.2	输入管线的组织方式	10
7.3	训练现状与主要限制	11
7.4	当前范围与后续工作	12
8	分布式训练、梯度同步与运行时整理	12
8.1	同步数据并行与梯度聚合	12
8.2	从逐参数同步到梯度 Bucketing	12
8.3	通信与计算的重叠	12
8.4	梯度压缩：实现路径与阶段结果	12

8.5	MPI 扩展性实验	13
8.6	Profile 结果：通信时间项拆分	14
9	受 IST 启发的参数归属与周期同步	14
9.1	集群验证与消融实验	15
	参考文献	16
	Glossaire	18
A	GEMM 实验环境、工作负载与搜索协议	18
A.1	实验环境	18
A.1.1	硬件平台	18
A.1.2	软件环境	18
A.1.3	测量协议	18
A.2	单线程调优使用的工作负载	19
A.3	两轮搜索协议	19
B	16 线程配置下的调度与同步开销诊断	19
C	OpenMP blocked / packing 参数筛选补充	20
D	AVX2 单线程调优结果汇总	21
E	AVX2 blocked-size 候选集合的推导	22
E.1	m_r 与 n_r 的实现相关推导	22
E.2	k_c 的缓存模型与候选生成	23
E.3	单线程基础 m_c/n_c 候选生成	25
F	线程感知 $kCacheBlockM/N_c$ 调优热图补充	27
G	固定方阵上的线程扩展补充实验	27

1 引言

1.1 项目背景与研究对象

神经网络系统的实现不仅涉及模型结构本身，也涉及训练运行时、数值计算内核以及数据输入路径等多个层面。以图像分类任务为载体，从零实现一套可训练的神经网络程序，有助于在统一代码基础上考察前向传播、反向传播、参数更新与实验执行流程之间的关系，并为后续的性能测量与实现比较提供可控环境。

本报告讨论的对象是一个以 C++ 为基础实现的神经网络实验系统。该系统当前覆盖了多层感知机 (MLP, multi-layer perceptron) 与卷积神经网络 (CNN, convolutional neural network) 两类模型路径，同时包含自动微分运行时、矩阵乘法与卷积计算后端、数据加载接口，以及面向分布式训练实验的同步机制。本文关注的是这些组成部分在当前实现范围内的组织方式、扩展方向与实验用途，而非将其表述为通用深度学习框架或稳定完备的训练平台。

1.2 前期基础与本阶段扩展

前期工作已经完成了基于 MNIST [1] 的多层感知机训练基础，包括反向模式自动微分、显式计算图、基础矩阵计算核心与完整训练循环。该部分工作提供了可运行的功能基线，也为后续比较不同计算实现、扩展卷积路径以及观察运行时开销提供了统一的实验起点。

在此前基础上，本阶段将系统从单一的 MLP/MNIST 原型扩展为能够承载更多技术路径的实验对象。扩展内容主要包括：加入可配置的 CNN 路径，以支持卷积模型实验；围绕通用矩阵乘法 (GEMM, general matrix multiplication) 与卷积相关算子进行实现比较与优化；整理自动微分运行时，以便更清晰地区分框架层开销与算子层开销；引入 Tiny-ImageNet 数据集路径，用于评估当前实现对更高负载图像输入与训练流程的适配范围；并加入基于消息传递接口 (MPI, message passing interface) 的同步数据并行实验路径，用于考察分布式梯度同步在当前实现中的组织方式与边界。

1.3 报告范围与后文内容

本文后续内容围绕上述扩展依次展开。首先讨论 GEMM 相关实现及其比较方法，并据此说明后续实验中复用的矩阵计算配置；随后介绍卷积路径与 CNN 结构在当前代码中的实现范围；在此基础上，再说明 Tiny-ImageNet 数据集接入、输入管线与训练支撑代码的组织方式；最后讨论 MPI 分布式同步路径和自动微分运行时整理的相关内容。附录部分补充若干诊断实验与参数推导过程，用于支持正文中的实现分析与实验解释。

2 GEMM 矩阵乘法算子的优化

在当前系统中，GEMM (general matrix multiplication, 通用矩阵乘法) 是核心底层计算算子，同时用于 MLP 的全连接层和卷积路径中的 `im2col + matmul` 方案。因此，其实现效率会直接影响 CNN、Tiny-ImageNet 和分布式训练实验的总体耗时。

本章围绕项目中的多种 GEMM 路径展开实现、比较与优化，包括 `blas` (OpenBLAS)、`omp` (上学期实现的基础 OpenMP 版本)、`omp_blocked`、`omp_blocked_packab`、`omp_gotoblas_avx2` 和 `omp_gotoblas_avx512`。目标是结合 OpenMP 并行、cache blocking (缓存分块)、packing (数据打包) 和 GotoBLAS 风格分层加 SIMD 微内核，使 GEMM workload 更适配当前设备。

具体而言，本章关注的问题包括：基础 OpenMP 并行能带来多少收益，cache blocking (缓存分块) 与 packing (数据打包) 是否能够改善访存行为，GotoBLAS 风格的分层结构在本项目的 workload (工作负载) 上是否有效，以及 AVX2/AVX-512 微内核在不同矩阵形状和线程数下表现是否稳定。相关结果主要用于形成后续实验中复用的 `matmul` 实现配置。

2.1 实验环境

除特别说明外，本章后续实验均使用附录 A 中列出的硬件环境、软件配置、测量协议和 CPU 绑定设置。GEMM 比较默认采用 1 至 8 线程配置，对应 8 个物理核心，并启用 `OMP_PROC_BIND=true` 与 `OMP_PLACES=cores`；只有 SMT/超线程对照实验使用 16 线程配置。

2.2 线程预算与实验协议

在正式比较各 GEMM 实现之前，先通过一组 SMT 对照实验确定线程预算。对照实验的结论是：16 线程（SMT）配置在 OpenBLAS 和基础 OpenMP 实现上均出现性能退化，主要原因是调度与同步等待开销在超线程共享的物理执行单元上被放大（详见附录 B）。

因此，后续所有实验均沿用这一基准配置，以避免 SMT 配置下的调度波动混入 backend 比较。施加亲和性约束后的线程扩展性对比见附录 B。

2.3 OpenMP 分块与打包技术

相对于 omp 基线，本节关注 **分块（Blocking）** 与 **数据打包（Packing）** 在并行执行时能在多大程度上进一步改善各线程的工作局部性。

从时间局部性（temporal locality）的角度看，blocking 的目的，是在子块被逐出前尽可能复用已进入 cache 的 A 、 B 与 C 子块。否则，尚未充分利用的数据就会被后续访问的新数据替换。

packing 则是在 blocking 之外，将后续需要反复读取的面板（panel）或子块复制到连续缓冲区中并重新排列。这样可以用一次额外复制换取更规则的顺序访存、更少的 stride 访问和更可控的 cache residency（缓存驻留），从而减少 cache / TLB miss 并提高带宽利用率。

具体调参与筛选过程见附录 C；在当前平台上，omp_blocked 更接近 L2-oriented blocking，而带 packing 的路径则更接近 L1-oriented 设计。

基于这一点，下面转向结构更完整的 GotoBLAS 风格路径，并继续考察其在当前设备与工作负载上的适配效果。

2.4 GotoBLAS 架构

前述 packing 优化已经改善了数据局部性，也为寄存器级计算提供了基础。在此之上，后续实现进一步引入微内核，以在 GotoBLAS 风格组织下更有效地利用寄存器层面的 SIMD。

2.4.1 三层分块模型

GotoBLAS 将 $C = A \cdot B$ （规模 $M \times K \times N$ ）分解为三层嵌套循环及最内层微内核 [3, 4]，如图 1 所示。

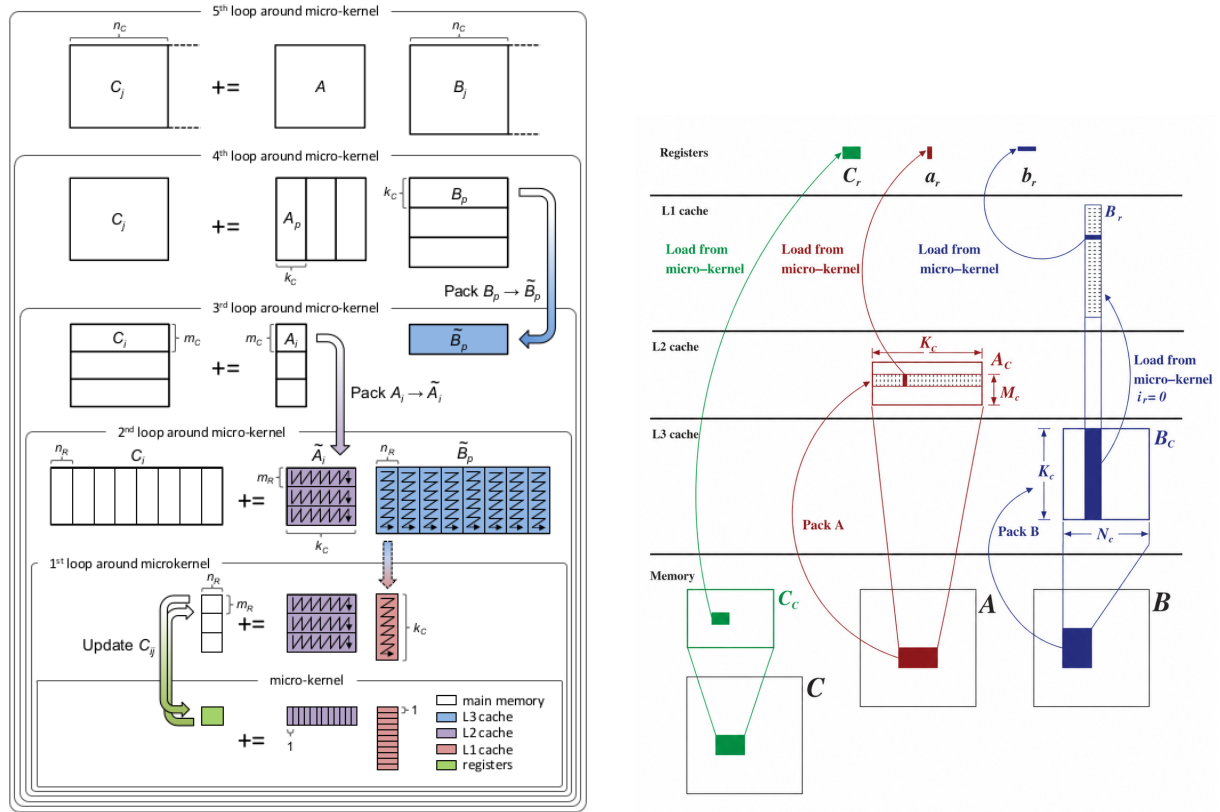


FIGURE 1 – 左：围绕微内核组织的 GotoBLAS/BLIS 风格 GEMM 分层结构，转引自 [3, Fig. 1]。右：BLIS/GotoBLAS 实现中数据移动与面板在内存层次中放置方式的示意图，转引自 [4, Fig. 4]。

外层三层 blocking 与 packing 的作用，是将工作集压缩并稳定驻留在合适的 cache 层级中；最内层 micro-kernel 则在寄存器层面最大化对这些 packed 数据的复用。

1. 外层 (N_c 维度)：沿 N 维以 N_c 为步长，将 B 的面板 ($K \times N_c$) 装入 LLC (L3 缓存)；
2. 中层 (K_c 维度)：沿 K 维以 K_c 为步长，将打包后的 B 面板 ($K_c \times N_c$) 装入 L2 缓存；
3. 内层 (M_c 维度)：沿 M 维以 M_c 为步长，将打包后的 A 面板 ($M_c \times K_c$) 装入 L1 缓存；
4. 微内核 ($M_r \times N_r$ 寄存器 tile)：最内层操作，在 SIMD 寄存器中累积 $M_r \times N_r$ 的 C 块。

需要说明的是，这一层次化模型默认各层块都能形成足够规整的 tile；但当 M 很小或线程数较高时，沿 M 维切分后每个线程实际分得的行数可能低于微内核高度 M_r 。此时宏块与寄存器块无法充分铺满，程序会更频繁地落入边界处理 (remainder loop)，从而削弱三层分块本应带来的缓存复用与并行效率。

缓存块参数按面板容量估算： A 面板 $M_c \times K_c \times 4$ 字节对应 L1 (32 KiB/核)， B 面板 $K_c \times N_c \times 4$ 字节对应 L2 (1 MiB/核)。当前默认参数为 $K_c = 384$ 、 M_c 有效行数 = 64 (倍率 8，基于 $M_r = 8$)、 $N_c = 448$ ；这些参数在第 2.5 节中继续做实验筛选。

2.5 GotoBLAS 调优与多线程并行策略

参数搜索以项目中的神经网络 GEMM 形状为对象。由于 $(M_r, N_r, K_c, M_c, N_c)$ 的组合空间过大，本文采用分阶段筛选：先缩减候选，再对保留形状做单线程实验。

2.5.1 微内核设计

当前 AVX2 单精度微内核沿 N 方向按 8 列组织，以匹配 YMM 向量宽度并减少边界处理。对 float32 路径而言，256 位 YMM 寄存器一次可覆盖 8 个 float，因此本实现优先筛选 N_r 为 8 倍数的形状。

参数搜索协议、 $(M_r, N_r, K_c, M_c, N_c)$ 搜索空间的缩减过程，以及分 workload 的详细结果统一放入附录 A、D 和 E。正文这里只保留后续多线程实验需要用到的结论。

2.5.2 结果摘要

在单线程下， 4×16 与 8×8 是最有竞争力的两种微内核形状。前者在多种全连接层形状上更强，后者则在部分卷积相关 workload 上更稳。代表性的单线程形状对比图与各工作负载族的最佳配置汇总见附录中的图 12 与表 8。

相对于 OpenBLAS，主流形状 ($N \geq 64$) 的差距通常在 10 至 24% 之间；而在极小 N 的形状上，这一差距会扩大到 66 至 78%，因为边界处理会占据更大比例。为了在后续多线程实验中使用一组固定配置，AVX2 路径最终采用 `avx2_8x8` 作为主要折中方案。

2.5.3 最终 AVX2 配置与 AVX-512 简介

基于上述单线程折中结果，后续多线程实验固定使用如下 AVX2 配置：

$$\text{avx2_8x8}, \quad K_c = 384, \quad M_c = 64, \quad N_c = 448$$

其中 $M_c = 64$ 为有效行数（代码常量 `kCacheBlockM` = 8，有效值 = `kCacheBlockM` $\times$ $M_r = 8 \times 8 = 64$ ）。该配置作为多线程比较中的固定基线。

AVX-512 路径沿用同一 `float32/Scalar=float` 数据路径，并采用 `avx512_8x32` 微内核。最终选定：`avx512_8x32`, $K_c = 160$, $M_c = 64$, $N_c = 448$ (M_c 有效行数为 64； $K_c = 160$ 为实验调优值，通过 `MATMUL_KC=160` 在运行时传入，与 AVX2 路径的编译常量 `kCacheBlockK` = 384 性质不同)。AVX2 blocked-size 候选集合的详细推导见附录 E。

2.5.4 GotoBLAS 多线程并行策略与参数调优

单线程调优确定了最优内核形状和块参数。进入多线程场景后，当前实现继续使用运行时给定的 M_c, N_c, K_c 作为缓存分块参数，并在并行调度层引入随线程数变化的任务粒度：代码在 ic/M 维上根据线程数计算更细的 `task_m`，以便在小规模矩阵下产生更多任务，减少线程空闲。

2.5.5 线程感知块参数调优

在当前代码中，线程感知主要体现在调度粒度而非缓存块参数本身：进入并行区后，程序会根据线程数计算更细的 `task_m`。因此，后续强扩展性实验只使用最终选定的固定 blocked-size 配置。

本文后续强扩展性实验采用固定配置：AVX2 路径使用 `avx2_8x8`, $K_c = 384$, $M_c = 64$, $N_c = 448$ ，AVX-512 路径使用 `avx512_8x32`, $K_c = 160$, $M_c = 64$, $N_c = 448$ 。附录图 13 给出的线程感知 `kCacheBlockM`/ N_c 热图也表明，高性能区域仍集中在 `kCacheBlockM` = 8（即有效 $M_c = 64$ ）一行，而 N_c 方向相对较平。

2.5.6 强扩展性实验 (Strong Scaling)

强扩展性实验在固定问题规模下取 $T \in \{1, 2, 4, 8\}$ ，对比 AVX2 GotoBLAS、AVX-512 GotoBLAS 与 OpenBLAS 三种实现；测试集覆盖 9 个来自 MLP、CNN 卷积和方阵的工作负载。

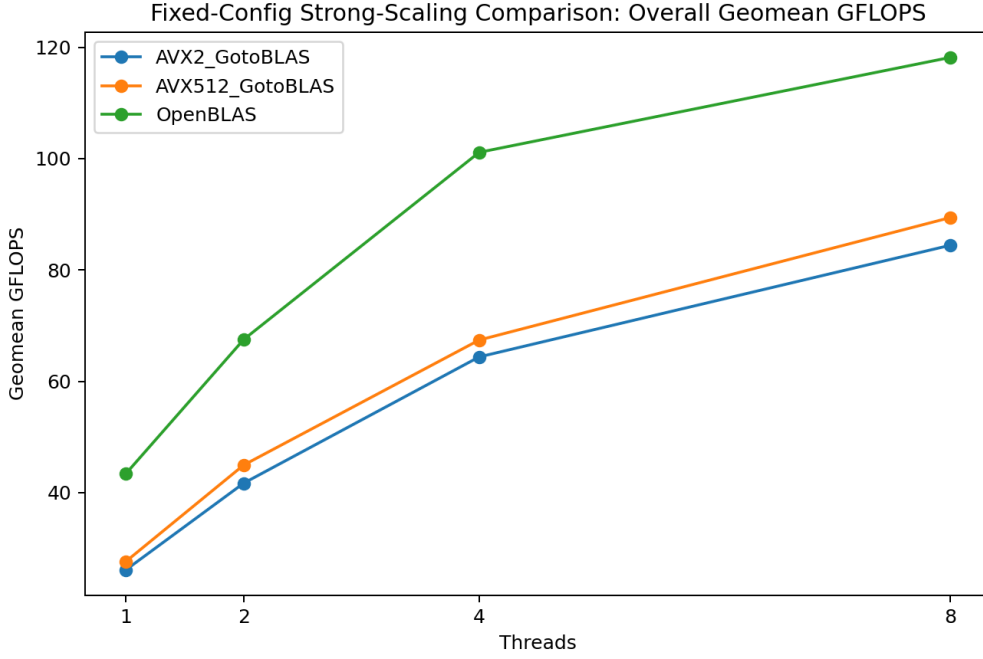


FIGURE 2 – 强扩展性实验：各线程数下的几何平均 GFLOPS（跨 9 个工作负载）。

图 2 给出了这 9 个 workload 的几何平均。OpenBLAS 仍保持最高的 GFLOPS，而 GotoBLAS AVX2 与 AVX512 的线程扩展更平稳：按这一口径，8 线程下两条 GotoBLAS 路径的 speedup 均约为 $3.23\times$ ，OpenBLAS 约为 $2.72\times$ 。若看逐 workload 结果，则多数情况下 OpenBLAS 仍领先，GotoBLAS 路径主要用于比较 blocking、packing 与 OpenMP 组织。

2.6 Unrolled+Prefetch 实现思路

在 AVX-512 路径完成基础调优后，代码又增加了一条 **unrolled+prefetch** 变体。该路径保留原有 blocked+packed 结构，只在 AVX-512 微内核及其对应的 packed panel 访问阶段加入循环展开（如 GEMM_UNROLL_4）和显式预取，从而进一步压缩最内层执行阶段的访存等待。它作用于 packed A/B 面板的连续访问路径，统计范围仍与前文相同。

固定方阵上的补充线程扩展实验主要用于观察合成尺寸上的行为，因此放入附录 G。

3 损失函数、优化器与训练循环

当前主训练入口的分类路径统一使用 **CrossEntropyLoss**：模型前向输出类别 logits，并由损失层在内部完成数值稳定的 softmax 与交叉熵计算。代码中也保留了 **MSELoss**，但这里不使用。

优化器方面，当前实现了 SGD、MomentumSGD [10] 和 AdamW [11] 三条路径。

对于 SGD，参数按当前梯度直接更新：

$$\theta_{t+1} = \theta_t - \eta g_t.$$

MomentumSGD 在此基础上引入速度项，用指数平滑的历史梯度稳定更新方向：

$$v_{t+1} = \mu v_t - \eta g_t, \quad \theta_{t+1} = \theta_t + v_{t+1},$$

并可按配置启用 Nesterov 修正。AdamW 则进一步维护梯度的一阶矩与二阶矩估计，并把 weight decay 从自适应更新中解耦。

训练配置在构图完成后根据命令行参数选择优化器，并在每个训练 step 中先清空梯

度，再在 backward 与可能的分布式同步完成后，按全局 batch 大小做缩放并执行参数更新。`Trainer::runEpoch(...)` 统一承担训练与评估两条路径：前者执行前向、损失计算、反向传播与参数更新，后者只保留前向与指标统计。这一训练循环随后也构成 CNN、ResNet 与分布式实验的共同基础。

图 3 汇总了三种优化器在 MNIST MLP 上的 20 epoch 训练曲线。

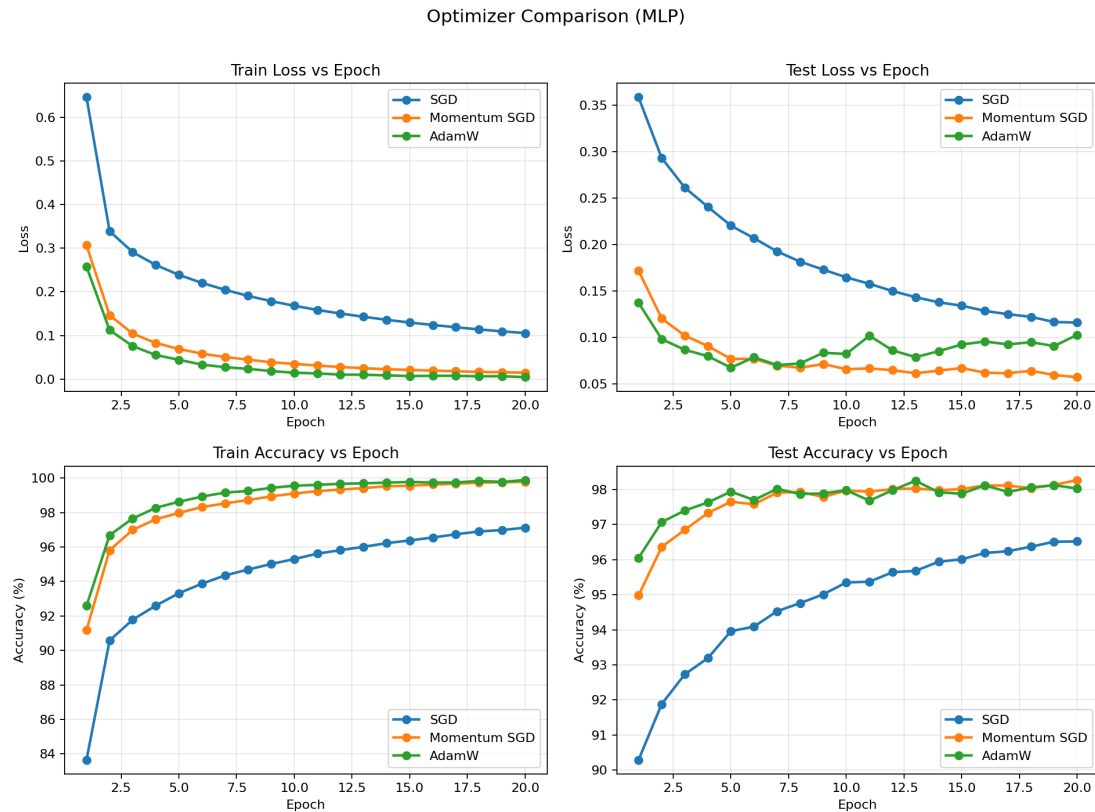


FIGURE 3 – 三种优化器在 MNIST MLP (20 epoch) 上的训练/测试损失与准确率对比。SGD 最终测试准确率 96.52%，MomentumSGD 98.27%，AdamW 98.03%。

SGD 的收敛最慢，最终测试精度也最低；MomentumSGD 在这里取得了最高的测试精度；AdamW 则达到接近的最终水平，但在训练初期收敛更快。

4 卷积算子的实现和优化

二维卷积在每个空间位置上对局部邻域做加权求和，并在整张图像上共享同一组权重。这样，它既能提取局部模式，又比直接作用于展平图像的全连接层使用更少参数；在当前实现中，这一算子采用 `im2col/GEMM/col2im` 路径 [7]，以复用现有的 GEMM 后端，基线配置下实际对应的是 OpenBLAS。

4.1 卷积层的初始实现方式

在 reference backend 中，`Conv2DLayer` 通过 `im2col` 将输入的局部感受野整理为矩阵各行，再把卷积前向改写为一次 GEMM：

$$\text{im2col}(X) \in \mathbb{R}^{(N \cdot H_{out} \cdot W_{out}) \times (C_{in} \cdot k_H \cdot k_W)},$$

$$W \in \mathbb{R}^{C_{out} \times (C_{in} \cdot k_H \cdot k_W)},$$

$$Y = \text{im2col}(X) \cdot W^T + b.$$

$$Y \in \mathbb{R}^{(N \cdot H_{out} \cdot W_{out}) \times C_{out}}, \quad \text{reshape}(Y) \in \mathbb{R}^{N \times C_{out} \times H_{out} \times W_{out}}.$$

这一实现复用了 `MathOps::matmul` 路径。在当前实现中，卷积层的总成本分布在 `im2col`、`GEMM`、输出重排、梯度计算 `dW/dX` 和 `col2im` 等阶段上；本节以前向主链路 `im2col/GEMM/col2im` 为主线，并在必要时把它放回前向与反向的整体成本中讨论。

4.2 卷积层并行化与切分策略

在当前项目的 CNN 实现中，卷积层采用 `im2col + GEMM` 路径，因此其性能并不只由 `GEMM` 决定；总成本时还包括 `im2col`、`col2im` 以及 `backward` 中的相关 `GEMM`。本节优化的目标不是单独加速某个 kernel，而是改善整条卷积链路的总吞吐：降低数据重排开销、改善多线程扩展性，并尽量减轻小规模任务中的并行负收益。

围绕这一目标，最核心的选择是并行切分维度。理论上，任务可以沿 `batch` 维、输出通道维、输出空间维（ $H_{out} \times W_{out}$ ）或归约维度划分，但本实现优先采用“输出元素独立”的原则，即由各线程负责互不重叠的输出块，以避免额外的锁和原子操作。对于前向路径，`im2col` 按样本和空间块划分，`GEMM` 直接复用底层矩阵乘法后端的并行策略；对于反向路径，`dW`、`dX` 和 `col2im` 也尽量围绕输出块组织，以降低冲突写入风险。

不过，切分粒度并非越细越好。当任务过短时，线程创建、同步和调度迁移等固定成本会迅速上升，这一现象在 MNIST 这类小尺寸输入上尤其明显。因此，实际策略仍需在可用并行度与单任务计算密度之间取得平衡。

4.3 实现细节与工程取舍

在工程实现层面，优化主要集中在三个方面。第一，改进内存访问连续性：通过调整 `im2col/col2im` 的循环顺序与索引计算方式，使内层循环尽可能落在连续地址上，从而提高缓存行利用率并降低地址计算开销。第二，优化临时缓冲区管理：在可行范围内复用中间 `buffer`，减少训练过程中的重复分配与释放，降低 `allocator` 抖动。第三，优化线程调度策略：优先采用更稳定的静态划分方式，在任务负载较为规则时减少 `runtime` 调度开销与线程迁移。

上述改动的共同特征是“不改变数学语义，只整理执行路径”。换言之，卷积前向与反向结果与基线实现保持一致，优化仅发生在计算组织、数据布局 and 并行调度层面，因此既便于验证正确性，也能与现有 `autodiff` 和优化器流程保持兼容。

4.4 对照与观测指标

这里的比较沿用前文一致的可复现实验范式：固定输入 `shape`、`batch`、硬件环境、编译选项和线程绑定策略。所考察的 `shape` 首先对应当前 CNN 实现在 MNIST 上的两层卷积及其 `backward`；线程数通常取 $\{1, 2, 4, 8\}$ ，并通过预热、重复测量、中位时间和标准差评估结果。

观测指标分别跟踪 `im2col`、卷积 `GEMM`、输出重排、`col2im` 和 `backward` 各阶段耗时，以区分主链路收益与整层端到端收益。

5 CNN 架构的实现

本章说明当前 CNN 路径如何接入现有训练框架。其核心思路是在尽量复用既有训练与计算基础设施的前提下，只调整引入 CNN 所必需的网络抽象层。

5.1 当前 CNN 结构与数据流

5.1.1 整体结构

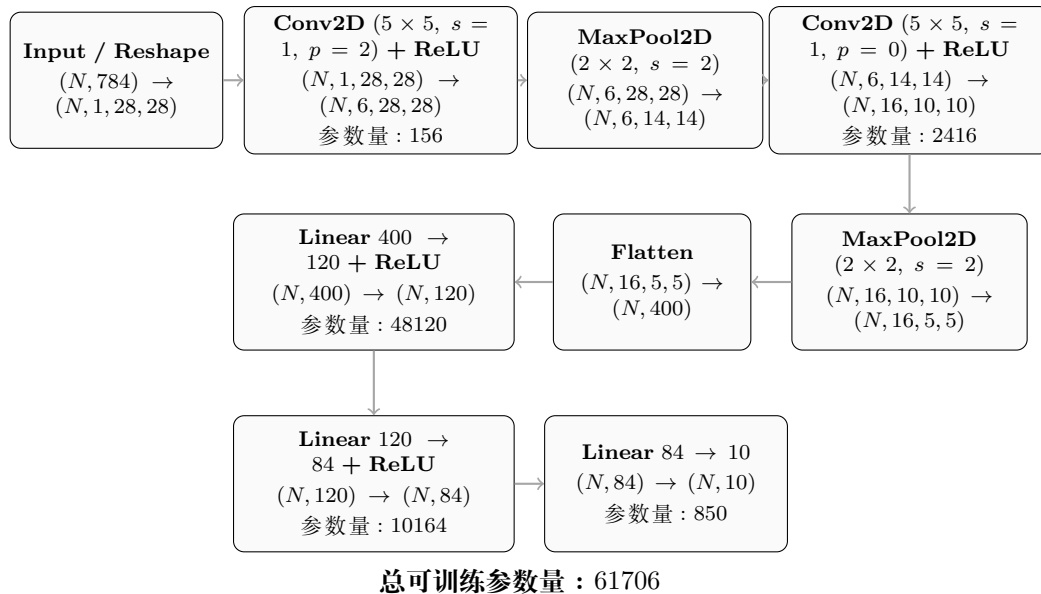


FIGURE 4 – 当前 CNN 路径的网络结构与各阶段 shape 变化。

新增组件各有不同作用。卷积层通过权重共享提取局部模式；在 MNIST 的 28×28 输入上， 5×5 卷积核已经能覆盖较有意义的局部区域，同时又不会把参数量抬得过高。池化层负责后续的空间降采样；这里选择 *MaxPooling*，是因为它可以在不增加可训练参数的前提下保留局部最强响应，并较快压缩后续层的计算规模。末端的全连接分类头则在特征已经压缩后，对剩余通道和空间信息做全局混合，并映射到类别 logits。

因此，这里选用受 LeNet [1] 启发的“两层卷积 + 两层池化 + 三层线性层”短结构，主要是为了在保持 MNIST 上代表性的同时，将训练和分析成本控制较低水平。

当前 CNN 路径先把每个样本从 $(N, 784)$ 解释为 $(N, 1, 28, 28)$ 的单通道图像，其 shape 变化如图 4 所示。

整套网络总计约 61706 个可训练参数。这个规模对应当前的 MNIST CNN 路径，并覆盖卷积、池化、特征展平和分类头四类计算。

5.1.2 前向数据流、存储表示与统一接口

该网络的前向路径与图 4 一致：输入先由 $(N, 784)$ 解释为 $(N, 1, 28, 28)$ ，随后依次经过两层卷积、两层池化、特征展平和三层全连接分类头。

这里的实现选择是继续使用统一的 **Matrix** 作为底层存储，而不单独引入 **Tensor4D**。整个项目统一采用单精度 **float**；逻辑上的四维张量 (N, C, H, W) 在内存中展平为二维矩阵 **Matrix**($N, C \times H \times W$)，再通过形状参数解释其语义。对某个元素 (n, c, h, w) ，其线性索引可写为

$$\text{idx} = c \times H \times W + h \times W + w.$$

这样做的优点是避免重写一套新的底层张量存储，并使卷积层可以直接复用现有矩阵运算接口；代价则是每一层都需要显式传递形状信息，并在实现中谨慎处理 **reshape** 与索引。完成两次卷积和池化之后，特征图会继续展平为 $(N, 400)$ ，再送入全连接分类头。

为避免在 MLP、CNN 和 ResNet 之间分别维护独立训练循环，这些模型统一暴露 **NeuralNetwork** 接口，只保留前向传播与可训练参数访问两项能力。这样，训练器、损失函数和优化器都可以保持不变，而 **CNNNetwork** 与 **ResNetNetwork** 只负责在“层实现 $\rightarrow$ 网络组合 $\rightarrow$ 训练入口”这一层次上补充各自的结构。

5.2 运行验证

模型	训练轮数	测试准确率	总 wall-clock 时间
MLP	1	90.57%	0.96 s
MLP	5	94.28%	3.39 s
CNN	1	96.08%	25.54 s

TABLE 1 – CNN 与 MLP 在 MNIST 上的粗粒度对比 (batch=256, seed=42)。最后一列给出一次运行中观测到的总 wall-clock 时间。

CNN 训练 1 个 epoch 即达到 96.08%，高于 MLP 训练 5 个 epoch 的 94.28%。但它的运行成本也明显更高：CNN 1 个 epoch 为 25.54s，而 MLP 1 个 epoch 为 0.96s、5 个 epoch 为 3.39s。

5.3 ResNet 结构扩展

在当前这条短 CNN 路径之外，一旦面向更大、更复杂的数据集，其模型容量就会很快显得不足。项目因此进一步加入了 **ResNetNetwork**，以引入更深且更易训练的卷积结构。ResNet [5] 的核心思想是残差连接：一个 block 会把计算拆成两条分支，主分支通过若干卷积产生残差项 $F(x)$ ，shortcut 分支则尽量直接传递输入 x 。在最简单的情形下，block 的输出写为

$$H(x) = x + F(x).$$

也就是说，block 不再从零学习一个完整映射，而只需要学习在输入基础上补上的修正量 $F(x)$ 。当输入输出的空间尺寸或通道数不一致时，shortcut 就不再直接等于 x ，而是改为一个投影 $P(x)$ ，此时输出写为

$$y = F(x) + P(x).$$

这种结构可以缓解深层网络训练中的梯度退化问题，使网络在增加深度时仍保持较好的可优化性。

当前 ResNet 扩展仍遵循统一的 **NeuralNetwork** 接口，因此可以直接接入现有训练链路，而无需为其单独编写一套训练循环。这样的设计既避免了重写训练框架，也便于直接检验残差结构与自动求导主线的兼容性。

5.3.1 结构与下采样策略

默认的 **ResNetConfig** 面向 MNIST 输入，输入形状为 $1 \times 28 \times 28$ ，类别数为 10。网络首先经过一个 stem 卷积层，然后进入三个 stage；各 stage 的通道数分别为 16、32、64，且每个 stage 含两个 residual block。对应的空间尺寸按 $28 \times 28 \rightarrow 28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7$ 变化，最后接一个全连接层输出 10 维 logits。

5.3.2 BasicBlock

当前 ResNet 的基本单元是 **BasicBlock**。如图 5 所示，它包含一条主分支和一条 shortcut 分支：主分支依次执行两层 3×3 卷积，而 shortcut 在与主分支结果相加后再经过最终激活 σ 。

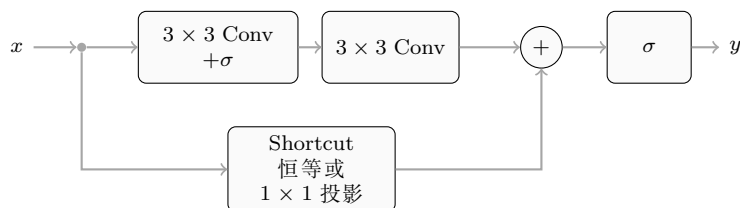


FIGURE 5 – 当前 ResNet 实现中一个 **BasicBlock** 的残差结构示意图。

记 $P(x)$ 为 shortcut 路径；当输入输出尺寸不变时，它是恒等映射，否则为一个 1×1 投

影。这里的 σ 表示 block 中使用的激活函数，在当前默认配置下为 ReLU。于是，一个 residual block 的计算可以写为：

$$\begin{aligned} F(x) &= W_2(\sigma(W_1x)), \\ y &= \sigma(F(x) + P(x)). \end{aligned}$$

各 stage 之间通过 stride 为 2 的卷积完成下采样，并在 shortcut 路径上同步使用 1×1 projection，以保证主分支与 shortcut 在空间尺寸和通道数上保持一致。该 ResNet 扩展已经接入代码，并显式处理了恒等 shortcut 与投影 shortcut 两种情况。不过，这一部分尚未展开系统性的超参数调优与深入 benchmark。若要继续面向更复杂数据集提升性能，后续仍需进一步补充 BatchNorm、全局平均池化和更系统的训练配置。

6 基于 Optuna 的超参数自动搜索

除手工调参外，项目还引入 Optuna [6] 组织自动化超参数搜索。其动机在于：早期 MNIST 实验已经表明，学习率、batch size、激活函数和优化器会显著影响收敛行为；进入 CNN 路径后，若继续依赖手工枚举，实验成本会迅速上升。因此，这里将 HPO 视为训练流程的自然延伸，而不是后续附加的脚本工具。

在实现上，每个 trial 独立执行一次训练，并以 `test_acc` 作为目标值。评价指标既可以取训练过程中的最佳测试精度，也可以取最后一个 epoch 的测试精度，即

$$\text{score} = \begin{cases} \max_e \text{test_acc}_e, & \text{metric=best,} \\ \text{test_acc}_E, & \text{metric=last.} \end{cases}$$

对应的 study 采用最大化方向。

搜索空间覆盖最敏感的训练参数：学习率、batch size、激活函数、初始化方式和优化器。优化器部分采用条件搜索，以避免无意义组合，并把 trial 预算集中到更有效的参数区域。

实验结果会连同参数配置和运行上下文一并保存。执行层面同时支持进程内并行与 MPI worker 模式。当前保留的一组 CNN HPO 结果中，40 个 trial、每个 trial 训练 3 个 epoch 的设置达到了 `best test_acc = 0.9891`；作为对照，默认手动配置在同样预算下可达到 98.45%。因此，最优 HPO trial 的提升幅度并不大，约为 +0.46 个百分点。

7 Tiny-ImageNet 数据集迁移与输入管线

在 CNN 路径和分布式训练接入后，实验对象从 MNIST 扩展到 Tiny-ImageNet，即 ImageNet 的一个缩小变体 [9]。这一迁移不仅扩大了输入规模，也引入了 JPEG 图像解码、卷积训练和分布式同步等更完整的系统负载，因此能够更充分地暴露当前实现的适用范围与瓶颈。

7.1 迁移动机与数据特征

MNIST 足以验证 MLP、损失函数和自动求导等基础路径，但不足以覆盖 JPEG 图像加载、CNN 计算和 MPI 同步等更高负载场景。相比之下，Tiny-ImageNet 提供了约 100000 张训练图像和约 10000 张验证图像，更适合评估当前实现对更复杂输入管线与训练流程的适配能力。当前实验采用训练集与验证集分离的设置，而未在主路径中接入独立测试集。

7.2 输入管线的组织方式

为适应 Tiny-ImageNet 的图像输入，系统不再沿用 MNIST 中“先整体载入、再按索引切分”的单一路径，而是在加载器与底层数据源之间引入 BatchSource 抽象，用统一接口描述样本规模、输入维度以及按索引装载 batch 的过程。在 MNIST 场景下，这一抽象可以退化为从已加载矩阵中复制若干行；在 Tiny-ImageNet 场景下，则改为在训练过程中按 batch 读取并解

码 JPEG 图像。加载器会复用 batch 缓冲区，并维持统一的索引与 shuffle 逻辑。

7.3 训练现状与主要限制

当前 Tiny-ImageNet 路径已经接入加载逻辑和训练循环。本节提到的评估对应训练集与验证集分离的训练/评估设置。

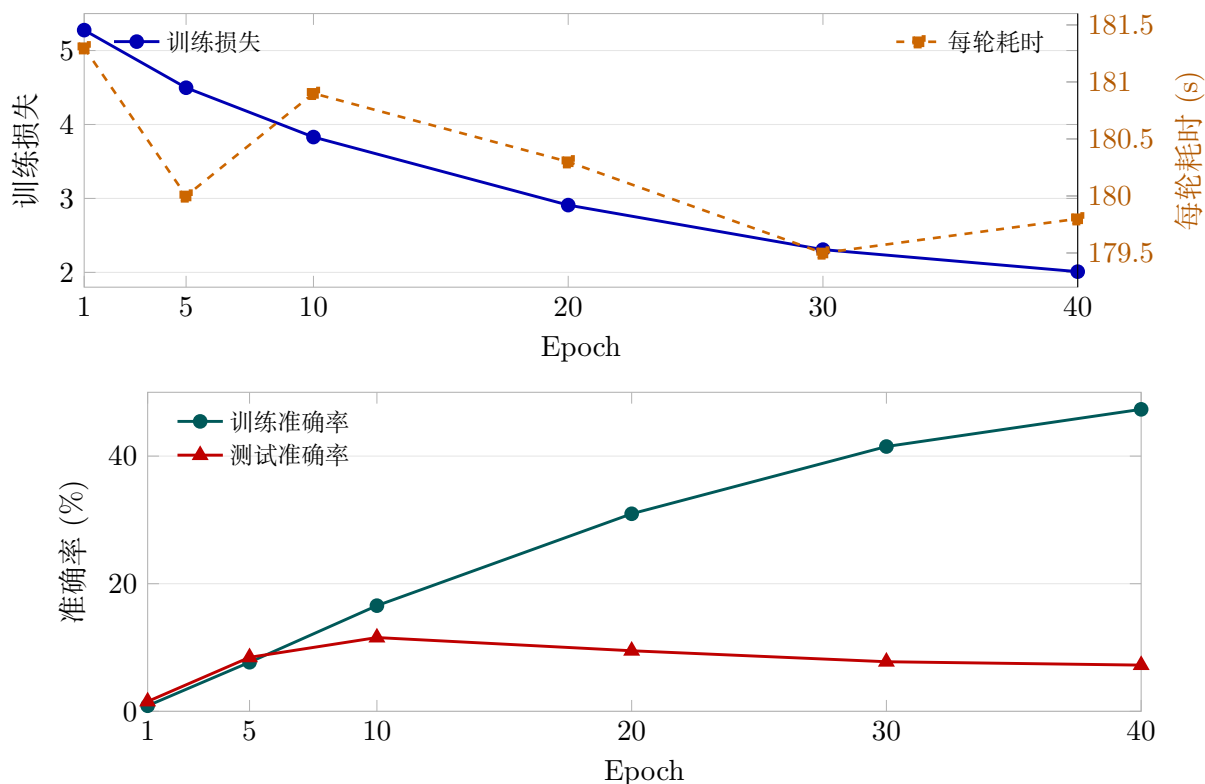


FIGURE 6 – Tiny-ImageNet 40 epoch 训练曲线摘要 (world_size=4, batch=128, per_param 同步)。上图展示训练损失与每轮耗时，下图展示训练准确率与测试准确率。

图 6 显示，训练准确率持续上升至 47%，但测试准确率在第 5 epoch 后即开始回落，第 40 epoch 仅 7.24%，表明在当前 LeNet 风格 CNN 结构 [1] 和无数数据增强的配置下，模型在 Tiny-ImageNet 上出现严重过拟合。

7.4 当前范围与后续工作

目前，这一路径已经能够稳定完成 Tiny-ImageNet 的数据装载与训练执行，但仍有明显限制。输入管线尚未覆盖异步预取、解码重叠、缓存和更完整的数据增强；模型侧则仍需继续检查 im2col/col2im、卷积 backward 以及卷积与 GEMM 之间的数据重排成本。后续工作仍需并行推进输入系统与模型训练，以提升 Tiny-ImageNet 的测试准确率。

8 分布式训练、梯度同步与运行时整理

本节介绍当前项目中的分布式训练路径：从同步数据并行出发，再讨论为降低同步开销而引入的运行时重组。重点不是证明多 rank 已能运行，而是分析通信粒度、计算与通信重叠以及梯度压缩会如何改变一次分布式训练 step 的真实成本。

8.1 同步数据并行与梯度聚合

当前分布式训练采用同步数据并行：逻辑上的模型仍然只有一套，但它会被复制到每个 rank 上，从而让多个 batch 可以在同一节点或多个节点上并行处理，而每个 rank 只负责自己的本地数据分片 [8]。每一步结束后，这些模型副本再通过梯度聚合重新同步，使它们随后执行同一组参数更新。设第 r 个 rank 在某一步得到的梯度和为 g_r ，则全局梯度先通过归约得到

$$g_{\text{sum}} = \sum_{r=0}^{\text{world_size}-1} g_r.$$

相应的全局 batch 大小为

$$B_{\text{global}} = \sum_{r=0}^{\text{world_size}-1} B_r.$$

因此，用于参数更新的全局平均梯度写为

$$g_{\text{global}} = \frac{1}{B_{\text{global}}} g_{\text{sum}} = \frac{1}{B_{\text{global}}} \sum_{r=0}^{\text{world_size}-1} g_r.$$

所有 rank 随后共享同一个平均梯度更新参数，因此模型副本在每一步后仍保持一致。真正的难点不在数学定义，而在于同步组织方式：逐参数阻塞式归约会引入大量小规模通信开销。

8.2 从逐参数同步到梯度 Bucketing

为降低固定通信开销，系统进一步引入梯度 bucketing：先将多个参数梯度打包到连续缓冲区，再以桶为单位执行归约。这样做并不改变训练语义，只是把同步粒度从“单个参数”提升到了“参数组”，从而减少集体通信调用次数。

8.3 通信与计算的重叠

在 bucketing 稳定后，系统又增加了通信与计算重叠路径：当某个桶内的梯度全部就绪时，即可在 backward 尚未结束时提前发起异步归约。这样，同步时机从“backward 结束后统一执行”前移到“梯度就绪后尽早执行”，从而为隐藏部分通信等待提供了可能。

8.4 梯度压缩：实现路径与阶段结果

梯度压缩 [12] 的目标，是在尽量保持收敛稳定的前提下降低同步成本。早期路径是在训练器侧、同步完成后再做本地压缩；它会改变梯度表示，却不会减少 AllReduce 的通信字节量，

因此后来被前移到了通信边界。

在本次迭代中，压缩逻辑被前移到分布式同步运行时，形成

`compress → AllReduce → decompress → step`

的执行框架，并统一接入 `bucketed` 与 `overlap_bucketed` 两条路径。配套实现引入了编解码钩子和分解计时指标，以区分编码、通信与解码开销；初版量化核因开销偏高，随后被更轻量的位运算实现替换。与此同时，压缩触发策略也从“按 bucket 间隔”修正为“按 step 间隔”，其中 `interval=8` 在当前平台上给出了较好的开销与收益折中。

在 `world_size=2`、MNIST、MLP(128)、`batch=64`、`seed=42` 条件下，三次重复实验的中位数结果如下：

- **bucketed** : fp16+i8 相对 baseline 的 `avg_step_time_ms` 为 +1.84%，`samples_per_s` 为 -1.80%，`test_acc` 为 -0.02%；
- **overlap_bucketed** : fp16+i8 相对 baseline 的 `avg_step_time_ms` 为 +1.58%，`samples_per_s` 为 -1.54%，`test_acc` 为 -0.02%。

基于上述结果，当前阶段的推荐配置为：`--grad_compress 1 --grad_compress_mode fp16 --grad_compress_interval 8`。

需要强调的是：当前虽然已经完成通信边界压缩路径的落地，但通信 payload 仍使用 float 缓冲区。因此，这里的结果主要说明压缩框架与调度策略已经成立；真实 payload 压缩通信的端到端收益仍需单独评估。

8.5 MPI 扩展性实验

表 2 展示了 MNIST CNN 在 `local_batch=64`、`seed=42` 下随进程数变化的运行结果。这里固定的是每个 rank 的 `local_batch`，因此 `global_batch` 会随 `np` 增大。

进程数	local_batch	global_batch	测试准确率	耗时 (s)	相对加速比
1	64	64	93.79%	31.66	1.00×
2	64	128	91.93%	19.49	1.62×
4	64	256	88.63%	11.98	2.64×

TABLE 2 – MNIST CNN (1 epoch, `local_batch=64`, `seed=42`) 的多进程扩展结果

进程数从 1 增至 4 时，耗时从 31.66s 降至 11.98s，加速比为 2.64×。该表中的准确率变化同时受到 `global_batch` 增大和每个 epoch 更新步数减少的影响。图 7 将实测加速比与理想线性加速对比。

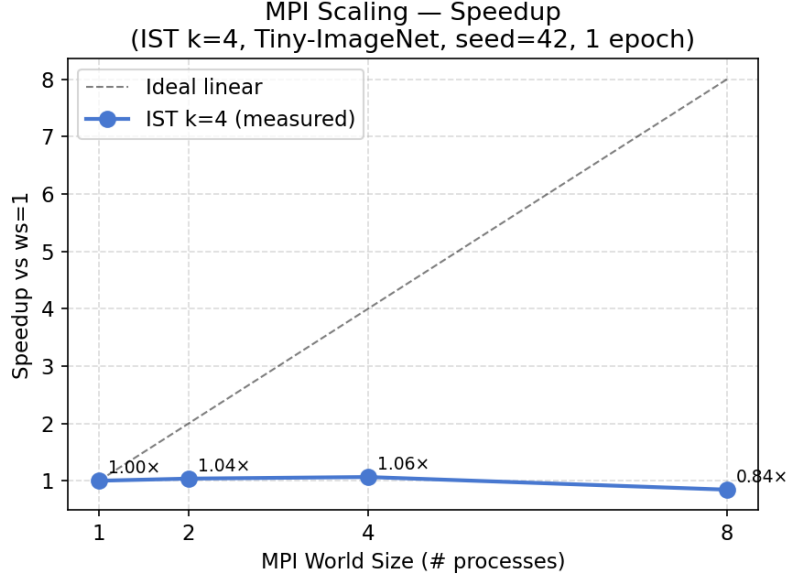


FIGURE 7 – MNIST CNN 随 MPI 进程数的实测加速比（虚线为理想线性加速，1 epoch，local_batch=64，seed=42）。

为进一步理解扩展性收益受限的原因，下面转向分项 profiling 结果。

8.6 Profile 结果：通信时间项拆分

当前 profiling 结果主要用于展示 fwd_bwd、sync_total 与 sync_wait 等时间项如何拆分，而不是直接给出严格可比的模式优劣。

同步模式	fwd_bwd (s)	sync_total (s)	sync_wait (s)
bucketed	7.57	1.60	1.60
overlap_bucketed	8.25	1.09	0.97

TABLE 3 – 同步 profile 中时间项的示例摘录

overlap_bucketed 当前仍以正确性验证为主，因此这里的结果主要用于定位通信与等待开销；真正的重叠收益仍需在完全匹配的配置下另行验证。

9 受 IST 启发的参数归属与周期同步

本文不复现标准 IST，而采用一种受其启发的参数归属方案。每个 rank 仍持有完整模型副本，并在本地执行完整前向与反向传播；区别在于参数更新阶段：完整参数向量被 ownership mask 划分给不同 rank，每个 rank 在一个同步窗口内只更新自己负责的参数位置，窗口结束后再通过参数增量合并恢复全局一致模型。

设模型参数展平后为 $\theta \in \mathbb{R}^d$ ，MPI 进程数为 R 。对第 r 个 rank，定义一个 0/1 ownership mask

$$m^{(r)} \in \{0, 1\}^d, \quad r = 0, \dots, R-1,$$

并要求每个参数位置恰好由一个 rank 拥有：

$$\sum_{r=0}^{R-1} m_i^{(r)} = 1, \quad i = 1, \dots, d.$$

在第 s 个同步窗口开始时，所有 rank 共享同一参数快照 θ_s 。窗口长度记为

$$K = \text{ist_local_steps}.$$

第 r 个 rank 在第 t 个本地 step 上，根据本地 mini-batch 计算完整模型梯度

$$g_{s,t}^{(r)} = \nabla_{\theta} \ell(\theta_{s,t}^{(r)}; \mathcal{B}_{s,t}^{(r)}),$$

但实际更新时只保留自己负责的参数位置：

$$\theta_{s,t+1}^{(r)} = \theta_{s,t}^{(r)} - \eta \left(m^{(r)} \odot g_{s,t}^{(r)} \right).$$

因此，mask 作用于参数更新阶段，而不是把前向或反向传播限制在显式子网中。

窗口结束后，第 r 个 rank 只提交自己负责部分的参数增量：

$$\Delta_s^{(r)} = m^{(r)} \odot \left(\theta_{s,K}^{(r)} - \theta_s \right).$$

由于不同 rank 的 mask 互不重叠，全局增量可通过求和式全规约得到：

$$\Delta_s = \text{MPI_Allreduce}_{\text{sum}} \left(\Delta_s^{(r)} \right) = \sum_{r=0}^{R-1} \Delta_s^{(r)}.$$

随后所有 rank 同步到同一组参数：

$$\theta_{s+1} = \theta_s + \Delta_s.$$

该过程不同于普通同步 SGD。普通同步 SGD 每一步都会聚合各 rank 对同一组参数计算出的梯度均值；而这里在一个同步窗口内，每个 rank 只改变自己拥有的参数块，非归属参数保持为窗口开始时的快照。因此，该方法应理解为一种降低同步频率的近似训练策略，而不是对逐步全量同步 SGD 的等价改写。

当前实现中，ownership plan 根据参数张量形状生成 mask：对类似全连接权重的二维参数按输出维划分，对一维参数或其他参数按元素划分。为避免参数长期固定绑定到同一 rank，程序支持通过 `--ist_resample_every` 在若干 epoch 后重新生成 ownership plan。这一机制可以缓解固定参数归属在非 IID 数据划分下带来的偏差风险，但不构成与同步 SGD 等价的理论保证。

需要注意的是，当前通信仍采用 dense delta 向量进行 `MPI_Allreduce`。因此单次同步的数据维度仍接近完整参数维度，尽管其中许多位置为零。现阶段收益主要来自把通信从每一步同步改为每 K 步同步一次；若未来要进一步降低单次通信量，则需要引入稀疏通信或仅传输归属索引。

9.1 集群验证与消融实验

在 zen.univ-lille.fr 集群上，我们首先进行了 IST 路径的可运行性验证与小规模消融实验。实验配置为 Tiny-ImageNet、ResNet、MPI world_size=2、batch=64、lr=0.01，优化器为 momentum_sgd。

图 8 给出了 seed=42、1 epoch 下的初步 K 值消融，比较 dense 路径、IST ($K=1$) 与 IST ($K=4$) 的验证准确率和每 epoch 耗时。该实验主要用于确认 IST 路径可以稳定运行，并观察不同同步窗口长度对耗时和早期准确率的影响；由于只包含 1 个 epoch 和单个随机种子，不能作为最终精度结论。

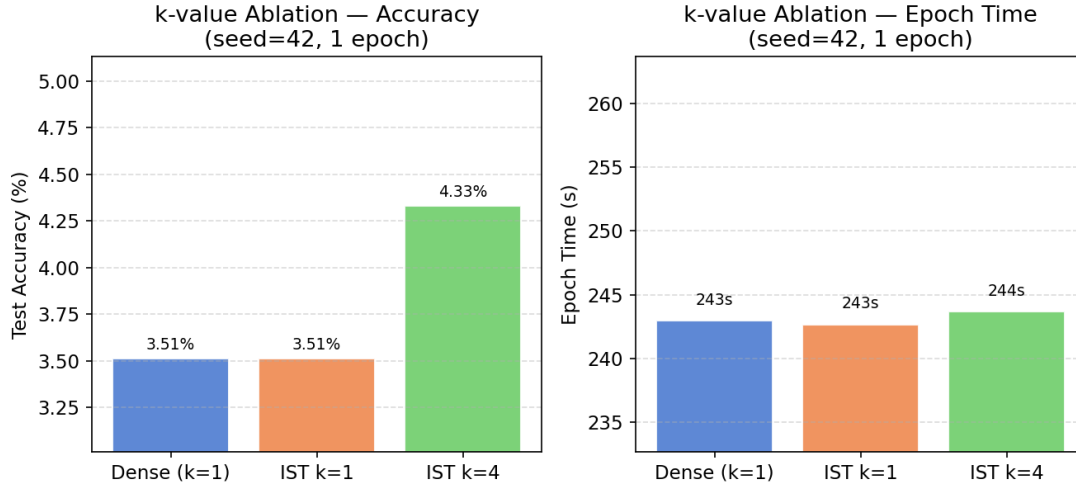


FIGURE 8 – dense、IST ($K = 1$) 与 IST ($K = 4$) 在 seed=42、1 epoch 下的验证准确率 (左) 与平均 epoch 耗时 (右) 对比 (Tiny-ImageNet, MPI world_size=2)。

隨後，图 9 比较了 dense 与 IST ($K = 4$) 在三个随机种子 {42, 43, 44}、2 epoch 运行中的平均每 epoch 耗时。在这组记录中，IST ($K = 4$) 的平均 epoch 耗时约为 dense 的 98.9%，说明降低同步频率在当前配置下带来了约 1% 的时间下降。不过该差异仍较小，且可能受到集群负载和随机初始化影响；因此这里更适合作为工程可行性验证，而不是强性能结论。

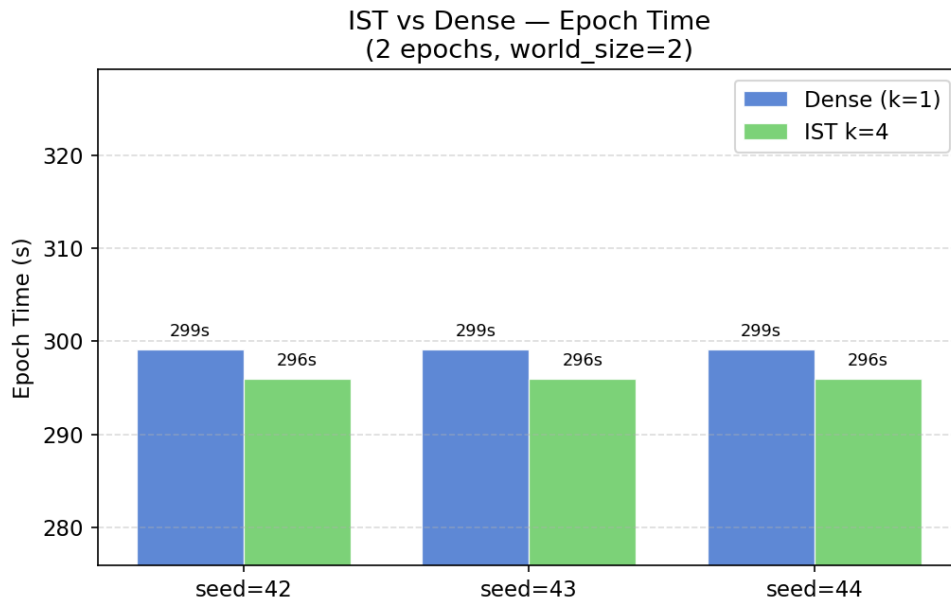


FIGURE 9 – dense 与 IST ($K = 4$) 在三个随机种子下的平均 epoch 耗时对比 (2 epoch, Tiny-ImageNet, MPI world_size=2)。在该小规模实验中，IST ($K = 4$) 的耗时略低于 dense。

Références

- [1] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. “Gradient-based learning applied to document recognition.” *Proceedings of the IEEE*, 86(11) :2278–2324, 1998.
- [2] Jack J. Dongarra et al. “A set of level 3 basic linear algebra subprograms.” *ACM Transac-*

- tions on Mathematical Software, 16(1) :1–17, 1990.
- [3] Field G. Van Zee and Tyler M. Smith. “Implementing High-Performance Complex Matrix Multiplication via the 3m and 4m Methods.” *ACM Transactions on Mathematical Software*, 44(1) :7 :1–7 :36, 2017.
 - [4] Tze Meng Low, Francisco D. Igual, Tyler M. Smith, and Enrique S. Quintana-Ortí. “Analytical Modeling Is Enough for High-Performance BLIS.” *ACM Transactions on Mathematical Software*, 43(2) :12 :1–12 :18, 2016.
 - [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. “Deep Residual Learning for Image Recognition.” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
 - [6] Takuya Akiba, Shotaro Sano, Toshihiro Yanase, Takeru Ohta, and Masanori Koyama. “Optuna : A Next-generation Hyperparameter Optimization Framework.” *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631, 2019.
 - [7] Yangqing Jia, Evan Shelhamer, Jeff Donahue, Sergey Karayev, Jonathan Long, Ross Girshick, Sergio Guadarrama, and Trevor Darrell. “Caffe : Convolutional Architecture for Fast Feature Embedding.” *Proceedings of the 22nd ACM International Conference on Multimedia*, pages 675–678, 2014.
 - [8] Alexander Sergeev and Mike Del Balso. “Horovod : Fast and Easy Distributed Deep Learning in TensorFlow.” *arXiv preprint arXiv :1802.05799*, 2018.
 - [9] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. “ImageNet : A Large-Scale Hierarchical Image Database.” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255, 2009.
 - [10] Ning Qian. “On the Momentum Term in Gradient Descent Learning Algorithms.” *Neural Networks*, 12(1) :145–151, 1999.
 - [11] Ilya Loshchilov and Frank Hutter. “Decoupled Weight Decay Regularization.” *International Conference on Learning Representations (ICLR)*, 2019.
 - [12] Dan Alistarh, Demjan Grubic, Jerry Li, Ryota Tomioka, and Milan Vojnovic. “QSGD : Communication-Efficient SGD via Gradient Quantization and Encoding.” *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.
 - [13] Binhang Yuan, Cameron R. Wolfe, Chen Dun, Yuxin Tang, Anastasios Kyrillidis, and Chris Jermaine. “Distributed Learning of Fully Connected Neural Networks using Independent Subnet Training.” *Proceedings of the VLDB Endowment*, 15(8) :1581–1590, 2022. <https://www.vldb.org/pvldb/vol15/p1581-wolfe.pdf>.

Glossaire

A GEMM 实验环境、工作负载与搜索协议

A.1 实验环境

A.1.1 硬件平台

组件	规格
处理器	AMD Ryzen 9 8940HX, x86_64 架构
物理核心/逻辑线程	16 物理核心, 32 逻辑线程 (每核 2 线程, SMT)
实验限制	通过 <code>taskset -c 0-7</code> 限制到 8 个 CPU, 并通过 <code>lscpu -e=CPU,CORE,SOCKET,NODE</code> 验证其物理核心映射
缓存层次	L1d 32 KiB/核, L2 1 MiB/核, L3 64 MiB (共享)
运行频率	锁定在 4.0 GHz (禁用 Turbo Boost)
内存	30 GiB

TABLE 4 – GEMM 实验硬件配置

A.1.2 软件环境

组件	版本/详情
操作系统	Fedora Linux 42
编译器	GCC 15.2.1, 编译选项 <code>-O3 -march=native</code>
BLAS 库	OpenBLAS (系统版本)
并行模型	OpenMP 5.0
向量指令集	AVX2 / AVX-512 (通过 <code>-march=native</code> 自动启用)

TABLE 5 – GEMM 实验软件配置

A.1.3 测量协议

测量协议如下:

- **预热 (Warm-up):** 每组测试前执行 5 次预热迭代, 以稳定指令缓存和数据缓存状态;
- **计时:** 程序内部循环 2000 次, 使用 `std::chrono::high_resolution_clock` 精确计时;
- **硬件计数器:** 通过 `perf stat -r 3` 外部重复 3 次, 采集指令数 (Instructions)、周期数 (Cycles)、上下文切换 (Context Switches)、CPU 迁移 (CPU Migrations) 及缓存缺失 (Cache Misses);
- **环境隔离:** 关键批次之间重启系统, 清空页缓存 (`drop_caches`), 通过 `cpupower` 锁定 CPU 频率, 以减少系统状态异质性。预热是稳定执行路径的主要机制; 重启和清空页缓存仅用于降低系统整体状态带来的跨批次干扰。

需特别指出: `perf` 的 `:u` (用户态) 修饰符会排除内核态上下文切换, 因此本文统一使用无修饰符的 `context-switches` 捕获完整调度事件。此外, AMD uncore L3 事件以 system-wide 方式采集, 短窗口下易受后台进程干扰, 故不用于定量比较; 本文改用 per-task 的 `cache-misses` 与 `cache-references` 作为聚合性内存压力指标。cache-misses 仅用于不同配置间的相对比较, 不用于精确区分 L1/L2/L3 来源。

CPU 余量与实验隔离。 GEMM 实验限制在可控的 CPU 核心预算内, 不占满全部逻辑线程, 以给操作系统、OpenMP 运行时、`perf` 采集和后台服务保留余量, 减少线程迁移、中断和非受

控资源竞争。本文据此区分两类配置：一类是限制在已验证物理核心预算内的推荐配置，另一类是 16 线程 SMT/超线程压力测试配置。

A.2 单线程调优使用的工作负载

参照项目 MLP 和 CNN 的实际层形状，定义了 4 个代表性工作负载族 (Workload Family)：

工作负载族	来源层	M	K	N
fc_forward_mainstream	MLP FC1 (hidden=128)	32	784	128
fc_head_small_n	MLP FC2 (output)	32	128	10
conv_dx_skinny_k	CNN Conv2 backward	3200	16	150
fc_wide_output	MLP FC1 (hidden=256)	32	784	256

TABLE 6 – 单线程调优使用的代表性工作负载 (batch=32)

A.3 两轮搜索协议

第一轮 (全局候选筛选)：对所有 6 种 AVX2 内核，在各工作负载上扫描多个 (M_c, N_c, K_c) 组合 (共 272 个候选)。每候选运行 1000 次取中位时间，并辅以 5 次 `perf stat` 采样记录硬件计数器。采用分阶段扫描策略：先固定 M_c, N_c 扫 K_c ，再固定最优 K_c 扫 M_c ，最后精扫 N_c ，使得性能变化可明确归因于单一参数轴。

第二轮 (局部精细化)：保留第一轮中在主流 FC 形状上表现最佳的 5 个 KernelShape/Kc 组合：8x8/Kc=288、8x8/Kc=384、4x16/Kc=192、4x16/Kc=256、4x16/Kc=320，在其最优区域周围加密 M_c, N_c 搜索网格 (每行约 75 个组合)，进一步精确定位最优参数。

B 16 线程配置下的调度与同步开销诊断

该诊断只用于确定实验协议，不作为本文的主要性能结论。

对照实验的核心证据来自 CPU 亲和性实验 (OpenBLAS、16 线程、 128×128 矩阵、5000 次迭代)：在自由调度条件下，上下文切换次数为 2,013、总指令数为 13.45 B；施加 `OMP_PROC_BIND=true`，`OMP_PLACES=cores` 后，上下文切换降至 14 (−99.3%)，总指令数降至 4.05 B (−69.9%)。这说明 16 线程配置的开销主要来自调度与同步等待，而非额外计算。

指标	自由调度	绑定核心	变化幅度
上下文切换 (CS)	2,013	14	−99.3%
总指令数	13.45 B	4.05 B	−69.9%
缓存缺失标准差	1.32%	0.06%	稳定约 22 倍

TABLE 7 – CPU 亲和性对 16 线程性能的影响 (128×128 矩阵，5000 次迭代)

据此，后续实验统一限制在 1 至 8 个物理线程并强制绑定核心，以避免 SMT 配置下的调度波动混入 backend 比较。

图 10 给出施加亲和性约束后的线程扩展性对比。

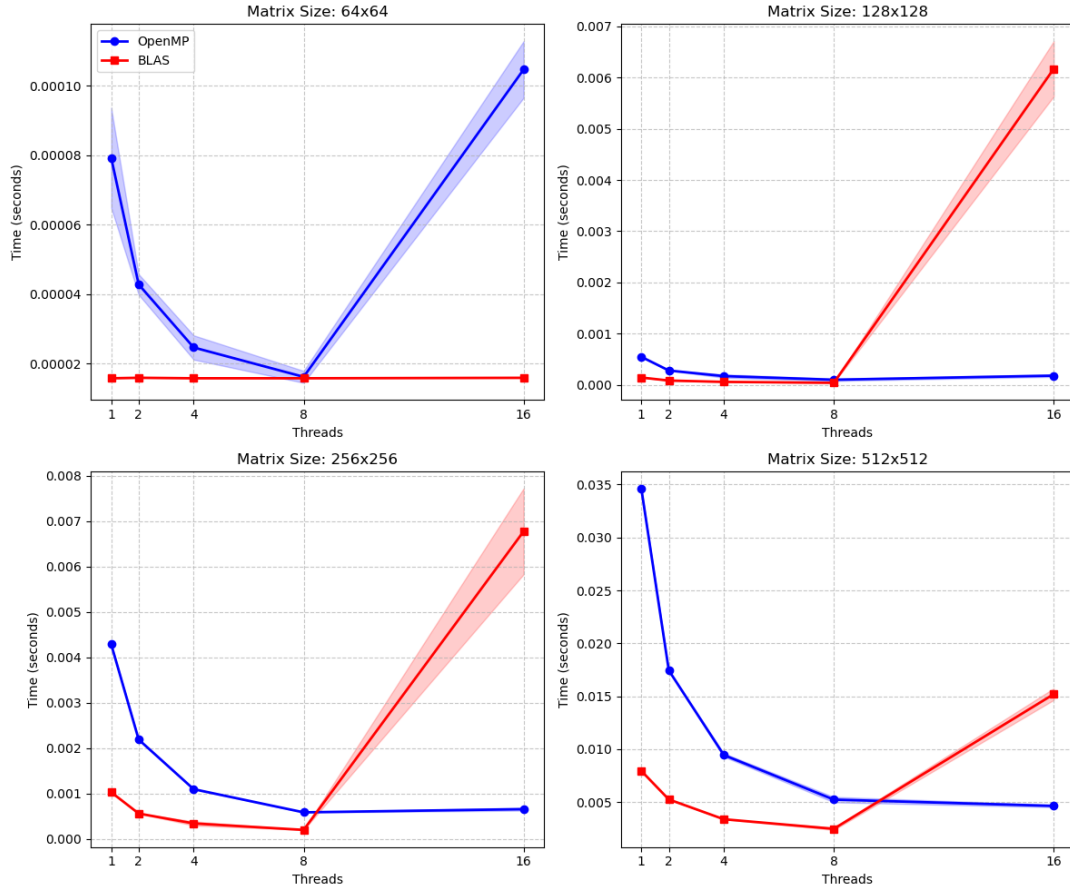


FIGURE 10 – 施加 CPU 亲和性约束后 (BLAS2) 的线程扩展性对比。在图中覆盖的 64×64 至 512×512 方阵上, 并在 8 个物理线程配置下, blas 相对于同图中的其他实现给出最低测得时间。

C OpenMP blocked / packing 参数筛选补充

本附录补充正文第 2.2 节之后 blocked 系列实验的完整 protocol。实验对象分为三条路径: `omp_blocked`、`omp_blocked_packb` 与 `omp_blocked_packab`。筛选过程分为两轮: 第一轮在 64、128、256 与 512 方阵上做小矩阵参数筛选, 统一测试 1、2、4、8 个物理线程, 默认每组重复 1000 次; 第二轮则对第一轮保留的少数候选在 1024 与 2048 上复测, 用于确认大矩阵下的稳定性, 并据此确定代表参数。

第一轮候选空间并非完全自由搜索, 而是按照正文中给出的 cache-resident 理论约束构造。`omp_blocked` 测试 `BlockSize` $\in \{16, 32, 48, 64, 128, 256\}$, 其中前 3 个取值对应偏 L1 的候选, 后 3 个取值对应偏 L2 的候选; `omp_blocked_packb` 只测试 `BlockSize` $\in \{16, 32, 48\}$, 因为其目标是考察偏 L1 的 packed-*B* 方案; `omp_blocked_packab` 则不再使用单一方块大小, 而是从 `PackM, PackN` $\in \{32, 48, 64, 96, 128\}$ 与 `PackK` $\in \{16, 32, 48, 64, 96\}$ 中保留满足 L1 footprint 约束的可行组合, 再在这些组合上做第一轮筛选。

第一轮结束后, 程序先对实验结果做聚合, 再以参考规模 (默认取最大筛选规模, 即 512) 下跨线程平均时间为依据, 自动选出进入第二轮的少数候选。图 11 仅展示 `omp_blocked` 第一轮筛选在四组方阵上的按线程平均汇总; 这一图的作用是观察 block size 的整体相对位置, 而不是直接宣布统一最优参数。对应的候选汇总与第二轮复测结果均已保留, 供后续复查。

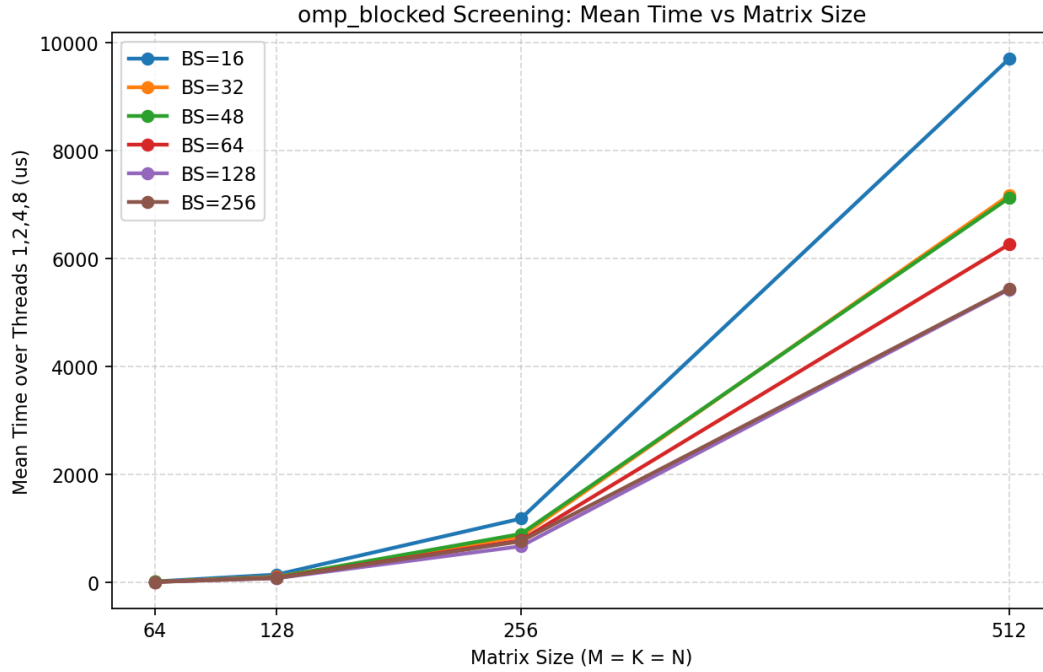


FIGURE 11 – `omp_blocked` 在 64 至 512 方阵上的小矩阵筛选汇总。图中对 1、2、4、8 个物理线程结果做平均后，可以看到 `BlockSize=64` 在较大方阵上更常落入较优区域，但不同矩阵尺寸与线程数下并不存在统一最优块大小。

与图 11 对应的 `omp_blocked_packb` 汇总 CSV 显示：在当前保留的 `BlockSize` $\in \{16, 32, 48\}$ 范围内，32 与 48 基本稳定优于 16；其中 64×64 上多由 32 领先， 128×128 上多由 48 领先， 256×256 与 512×512 上则常在 32 与 48 之间切换。对 `omp_blocked_packab` 而言，第一轮筛选后的第二轮候选集中在少数 L1-oriented 组合上，最终保留的代表参数为 $(\text{PackM}, \text{PackN}, \text{PackK}) = (32, 128, 16)$ 。

D AVX2 单线程调优结果汇总

图 12 给出了一个代表性全连接工作负载 `fc_forward_mainstream_nn / mlp_fc1_b32` ($32 \times 784 \times 128$) 上的结果。对每种微内核形状，图中仅保留其对应的最佳 `blocked` 参数组合。

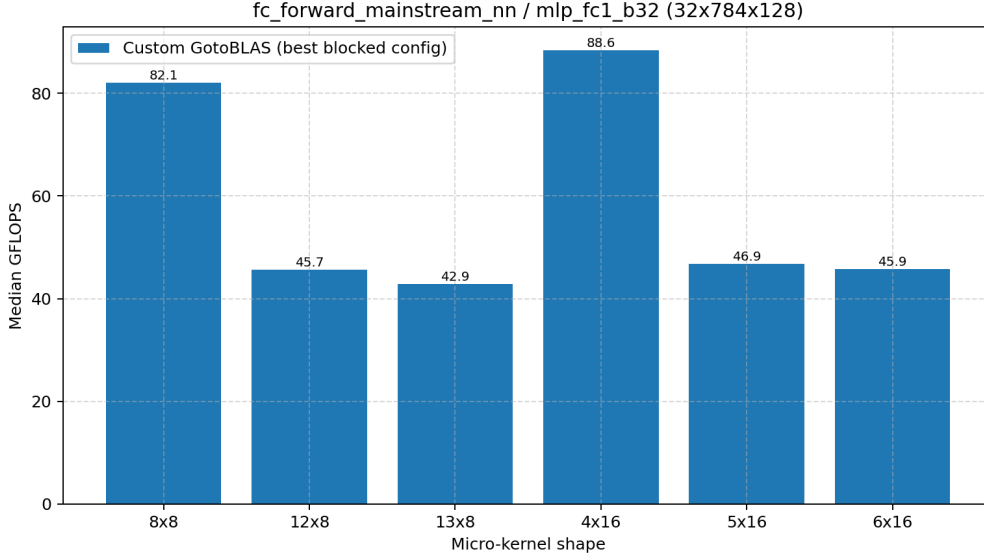


FIGURE 12 – 主流全连接工作负载 `mlp_fc1_b32` 上，不同 AVX2 微内核形状的单线程 GFLOPS 对比。横轴为 micro-kernel 形状，纵轴为该形状在最佳 blocked 参数下的中位 GFLOPS。

在该 workload 上，4x16 与 8x8 两种微内核形状明显领先：前者约为 88.6 GFLOPS，后者约为 82.1 GFLOPS，而其余形状大多仅落在 43 至 47 GFLOPS 区间。

工作负载族	最优内核	M_c	N_c	K_c	自定义 (μs)	OpenBLAS (μs)	差距
fc_forward (b32)	4x16	128	576	320	72.5	60.2	-17%
fc_forward (b64)	4x16	192	512	256	132.6	110.1	-17%
fc_head (b32)	4x16	—	—	—	4.76	1.14	-76%
conv_dx (b32)	8x8	160	640	288	744.3	647.3	-13%
fc_wide (b32)	4x16	128	768	256	136.5	122.2	-10%

TABLE 8 – AVX2 单线程调优：各工作负载族的最佳配置及与 OpenBLAS 单线程的对比

从表 8 可见，在主流形状上，该实现与 OpenBLAS 的差距通常在 10 至 24% 之间；而在很小的 N 形状上（如 `fc_head`），边界处理占比更高，差距会扩大到 66 至 78%。

E AVX2 blocked-size 候选集合的推导

本附录补充正文中 $(M_r, N_r, K_c, M_c, N_c)$ 搜索空间缩减的依据。

E.1 m_r 与 n_r 的实现相关推导

当前微内核在每次 p 迭代中执行 $C_r(i, :) \leftarrow C_r(i, :) + A_r(i, p) B_r(p, :)$ ，其中 $0 \leq i < m_r$ 。这里 $B_r(p, :)$ 被一次性装入一个 AVX2 向量寄存器， $A_r(i, p)$ 则通过标量广播参与 FMA。语义上的 $m_r \times n_r$ 小块保持不变；SIMD lane 对应的是 n_r 维度，寄存器方向因此与部分论文示例不同。

令 N_{vec} 表示一个向量寄存器可容纳的标量数。对 AVX2 `float32`，有 $N_{\text{vec}} = \frac{256}{32} = 8$ 。在参数优化语境中， n_r 至少需要满足向量化粒度约束，并通常取为 N_{vec} 的整数倍，即 $n_r = t N_{\text{vec}}$ ，其中 $t \in \mathbb{Z}_{\geq 1}$ 。若一个 $B_r(p, :)$ 在同一次迭代中由两个 YMM 寄存器共同承载， n_r 也可以取 16。本文后续只考虑 $t \in \{1, 2\}$ ，亦即 $n_r \in \{8, 16\}$ ，暂不讨论 $n_r = 24, 32$ 这类更宽的 n 方向微内核。

m_r 的约束来自吞吐隐藏和寄存器预算。BLIS 论文中关于微内核的基本条件可写为 $m_r n_r \geq N_{\text{vec}} L_{\text{vfma}} N_{\text{vfma}} = 8 \times 4 \times 2 = 64$ ，其中 L_{vfma} 是向量 FMA 的相关延迟，当前取 4 作为一阶建

模值； N_{vfma} 是每周期可发射的向量 FMA 数，当前取 2 作为一阶建模值，这给出了吞吐下界。

寄存器预算不能再写成与 n_r 无关的统一形式，因为当 $n_r = 8$ 时，每一行累加器只占 1 个 YMM；当 $n_r = 16$ 时，每一行累加器占 2 个 YMM。令 $t = n_r / N_{\text{vec}} = n_r / 8$ ，则累加器寄存器数为 $m_r t$ 。因此，寄存器预算至少应写成依赖 t 的形式：若一次迭代中只额外保留 1 个 B 向量临时寄存器，则有 $m_r t + 1 + \delta \leq N_{\text{reg}} = 16$ ；若进一步针对 $n_r = 16$ 采用更保守的建模，即认为两个 B 子向量都需要同时保留，则有 $m_r t + t + \delta \leq N_{\text{reg}} = 16$ 。其中 N_{reg} 是可用向量寄存器数， δ 是留给广播、地址生成和调度的余量，且 $\delta \in [1, 3]$ 。

因此必须分情况讨论。当 $n_r = 8$ ，即 $t = 1$ 时，吞吐下界给出 $m_r \cdot 8 \geq 64 \Rightarrow m_r \geq 8$ ，而乐观寄存器预算给出 $m_r + 1 + \delta \leq 16$ 。当 $\delta \in [1, 3]$ 时，可得到 $8 \leq m_r \leq 13$ ，因此当前建模下可保留 $m_r \in \{8, 9, 10, 11, 12, 13\}$ 。当 $n_r = 16$ ，即 $t = 2$ 时，吞吐下界给出 $m_r \cdot 16 \geq 64 \Rightarrow m_r \geq 4$ 。乐观寄存器预算变为 $2m_r + 1 + \delta \leq 16$ ，更保守的预算写法则为 $2m_r + 2 + \delta \leq 16$ 。按这组约束， $n_r = 16$ 时的可行 m_r 区间更窄；若取稳妥候选，则可写成 $m_r \in \{4, 5, 6\}$ ，而 $m_r = 7$ 只在“ $n_r = 16$ 且仅保留 1 个 B 向量临时寄存器”的较乐观预算下才可能保留。

本节给出的可行域按 $n_r/8$ 分情况约束：

$$\begin{cases} n_r \in \{8, 16\}, \\ m_r n_r \geq 64, \\ m_r \frac{n_r}{8} + 1 + \delta \leq 16 \quad (\text{较乐观写法；当 } n_r = 16 \text{ 时表示只保留 1 个 } B \text{ 向量临时寄存器}), \\ m_r \frac{n_r}{8} + \frac{n_r}{8} + \delta \leq 16 \quad (\text{较保守写法；对 } n_r = 16 \text{ 等价于同时保留全部 } B \text{ 子向量}). \end{cases}$$

据此，原始可行域中的 (m_r, n_r) 候选组合可写为

$$\{(8, 8), (9, 8), (10, 8), (11, 8), (12, 8), (13, 8), (4, 16), (5, 16), (6, 16)\},$$

其中 $(7, 16)$ 只在较乐观的寄存器预算下才可能保留，因此这里不将其放入默认候选集合。为控制后续搜索规模，本文后续固定保留的 (m_r, n_r) 候选集合为 $\{(8, 8), (12, 8), (13, 8), (4, 16), (5, 16), (6, 16)\}$ 。

E.2 k_c 的缓存模型与候选生成

B.2.1 基于 L1 的 k_c 候选中心

在给定 (m_r, n_r) 后， k_c 的角色是确定 packed $A_r \in \mathbb{R}^{m_r \times k_c}$ 与 $B_r \in \mathbb{R}^{k_c \times n_r}$ 的深度。本文先依据 L1 侧的面板占用关系构造候选中心，再结合对齐与 page-footprint 约束得到离散候选集合。

对 AVX2 float32，若以 cache line 为基本单位并记 L1 每路对应 1024B，则可把 A_r 与 B_r 面板的近似 set 压力分别写成与 $m_r k_c$ 、 $n_r k_c$ 成正比的量。简化后， A_r 方向的可用 set 预算上界可写为 $C_{A_r}^{\max} = \left\lfloor \frac{7}{1+n_r/m_r} \right\rfloor$ ，再由此得到候选中心 $k_c^{\text{center}} \approx \frac{1024 C_{A_r}^{\max}}{m_r}$ 。

k_c^{center} 在本文中只作为当前硬件上的候选中心。当前微内核在每个 p 迭代中先读取 $B_r(p, :)$ ，后广播 $A_r(:, p)$ 的标量；因此，L1 模型在本文中的作用是给出中心值与离散候选的生成起点。

B.2.2 基于 TLB / page-footprint 的 k_c 约束

仅由 L1 / set-mapping 关系确定 k_c 仍然不够，因为随着 k_c 增大，微面板与宏面板的 page footprint 都会同步扩大。若页大小为 P ，则微面板级 footprint 近似满足

$$\frac{m_r k_c S_{\text{data}}}{P} \lesssim E_{A_r, \text{TLB}}, \quad \frac{k_c n_r S_{\text{data}}}{P} \lesssim E_{B_r, \text{TLB}},$$

这类弱裁剪在本文中只作为提醒，因此后续改用宏面板级裁剪。为把 TLB 层实例化为可执行规则，这里暂定使用 $m_c = 128$ 、 $n_c = 512$ 作为 provisional panel volumes；它们只服务于当前轮次的 TLB-aware pruning。对 $P = 4096$ ，有

$$\text{pages}(A_c) \approx \left\lceil \frac{m_c k_c S_{\text{data}}}{P} \right\rceil = \left\lceil \frac{128 \cdot k_c \cdot 4}{4096} \right\rceil = \left\lceil \frac{k_c}{8} \right\rceil,$$

$$\text{pages}(B_c) \approx \left\lceil \frac{k_c n_c S_{\text{data}}}{P} \right\rceil = \left\lceil \frac{k_c \cdot 512 \cdot 4}{4096} \right\rceil = \left\lceil \frac{k_c}{2} \right\rceil.$$

当前机器的 L1 DTLB 和 L2 DTLB 条目数分别为 72 和 3072。由于 B_c 的页数对本节候选远小于 L2 DTLB 容量，真正有区分度的裁剪来自线程私有的 A_c 。为给栈、 C 块、代码和其他数据保留余量，这里取保守的有效预算 $E_{A,\text{eff}} = 0.75 \times 72 = 54$ ，并采用如下 pruning 规则：若某个候选使 $\text{pages}(A_c) > 54$ ，则将其从第一轮实验集合中剔除；若 $\text{pages}(A_c) \leq 54$ 且 $\text{pages}(B_c) \ll 3072$ ，则该候选通过当前 TLB 层检查。该层只承担近似过滤作用，不承担新的中心值生成。

B.2.3 综合 L1 与 TLB 约束的 k_c 候选生成

候选生成分两步进行。第一步，由 L1 模型围绕候选中心生成原始离散集合。若记 g_k 为 k_c 的实现对齐粒度，则有

$$\mathcal{K}_{L1}(m_r, n_r) = \text{Align}_{g_k}(\{\alpha k_c^{\text{center}}(m_r, n_r) : \alpha \in \mathcal{A}_k\}),$$

其中 g_k 由 packed panel 的步长与外层块循环粒度决定， $\mathcal{A}_k$ 表示围绕 1 的有限比例邻域。为控制第一轮搜索规模，这里取保守实例化 $g_k = 32$ 、 $\mathcal{A}_k = \{0.75, 1.00, 1.25\}$ 。

第二步，由 TLB / page-footprint 条件对 $\mathcal{K}_{L1}(m_r, n_r)$ 做一次近似裁剪，得到第一轮实验集合

$$\mathcal{K}_{\text{pruned}}(m_r, n_r) \subseteq \mathcal{K}_{L1}(m_r, n_r).$$

这里的 k_c^{center} 、 $\mathcal{K}_{L1}$ 和 $\mathcal{K}_{\text{pruned}}$ 都依赖于给定的 (m_r, n_r) 。后续实验可以对每个 (m_r, n_r) 分别搜索，也可以把各个候选集合并成一个并集候选池，但层次关系保持不变：L1 先生成，TLB 再裁剪。

对第 B.1 节最终保留的 6 组 (m_r, n_r) 候选，按 $C_{A_r}^{\text{max}} = \left\lfloor \frac{7}{1+n_r/m_r} \right\rfloor$ 与 $k_c^{\text{center}} \approx \frac{1024 C_{A_r}^{\text{max}}}{m_r}$ 计算，可得到表 9。表中先给出 $\mathcal{K}_{L1}(m_r, n_r)$ ，再给出经 TLB 层筛查后的 $\mathcal{K}_{\text{pruned}}(m_r, n_r)$ ；其中使 provisional A_c 页数超过 54 的偏大候选被实际剔除。

(m_r, n_r)	$C_{A_r}^{\text{max}}$	k_c^{center}	$\mathcal{K}_{L1}$	$\mathcal{K}_{\text{pruned}}$
(8, 8)	3	384.0	{288, 384, 480}	{288, 384}
(12, 8)	4	341.3	{256, 352, 416}	{256, 352, 416}
(13, 8)	4	315.1	{224, 320, 384}	{224, 320, 384}
(4, 16)	1	256.0	{192, 256, 320}	{192, 256, 320}
(5, 16)	1	204.8	{160, 192, 256}	{160, 192, 256}
(6, 16)	1	170.7	{128, 160, 224}	{128, 160, 224}

TABLE 9 – k_c 候选的生成与 TLB 裁剪结果

这些 $\mathcal{K}_{\text{pruned}}(m_r, n_r)$ 构成第一轮 k_c 搜索候选。它们来自“当前硬件上的中心值 + 32 对齐 + 有限比例邻域”，再经一层 page-footprint / TLB 过滤；例如 1.25 比例产生的 $k_c = 480$ 在 (8, 8) 这一行中因 provisional A_c 页数达到 60、超过有效 L1 DTLB 预算 54 而被剔除。

本节约束可概括为

$$\begin{cases} C_{A_r} + C_{B_r} \leq W_{L1} - 1, \\ C_{A_r} \leq \left\lfloor \frac{W_{L1} - 1}{1 + n_r/m_r} \right\rfloor, \\ k_c^{\text{center}} \approx \frac{1024 C_{A_r}}{m_r}, \\ \mathcal{K}_{L1}(m_r, n_r) = \text{Align}_{g_k}(\{\alpha k_c^{\text{center}}(m_r, n_r) : \alpha \in \mathcal{A}_k\}), \\ \mathcal{K}_{\text{pruned}}(m_r, n_r) \subseteq \mathcal{K}_{L1}(m_r, n_r). \end{cases}$$

E.3 单线程基础 m_c/n_c 候选生成

B.3.1 基于 cache 的 m_c/n_c 约束

m_c 与 n_c 建立在已经筛出的 (m_r, n_r, k_c) 候选之上：给定微内核形状和沿 k 方向的面板深度后， m_c 决定 $A_c \in \mathbb{R}^{m_c \times k_c}$ 的私有工作集高度， n_c 决定 $B_c \in \mathbb{R}^{k_c \times n_c}$ 的共享工作集宽度。本节据此把 cache / TLB 约束推进成单线程或弱并行情形下的基础候选集合。

A_c 在当前实现中按线程私有工作集处理， B_c 对应多线程路径中的共享读取面板，因此本节先按单线程基础候选讨论其体积。两者的字节规模分别为

$$|A_c| = m_c k_c S_{\text{data}}, \quad |B_c| = k_c n_c S_{\text{data}}.$$

基于 cache 层次，可写出最直接的尺度约束

$$m_c k_c S_{\text{data}} \lesssim \rho_A S_{\text{private}}, \quad k_c n_c S_{\text{data}} \lesssim \rho_B S_{\text{shared}}.$$

其中 $\rho_A, \rho_B \in (0, 1)$ 是用于保留冲突、代码和其他工作集余量的建模系数；这些关系式给出 $m_c^{\max}$ 与 $n_c^{\max}$ 的基础尺度上界。对当前机器，单线程基础建模中取每核私有 L2 容量 $S_{\text{private}} = 1$ MiB；为限制单个 B_c 面板的预算，再取保守的 effective shared-cache budget $\rho_B S_{\text{shared}} = 1$ MiB。再令 $\rho_A = 1/4$ ，则有

$$\begin{aligned} m_c^{\max}(k_c) &\approx \frac{\rho_A S_{\text{private}}}{k_c S_{\text{data}}} = \frac{256 \text{ KiB}}{4k_c} = \frac{65536}{k_c}, \\ n_c^{\max}(k_c) &\approx \frac{\rho_B S_{\text{shared}}}{k_c S_{\text{data}}} = \frac{1 \text{ MiB}}{4k_c} = \frac{262144}{k_c}. \end{aligned}$$

因此，cache 层在本节中的角色是给出随 k_c 变化的基础尺度上界 $m_c^{\max}(k_c)$ 与 $n_c^{\max}(k_c)$ ，后续离散候选将在这些上界附近生成。

B.3.2 基于 TLB / page-footprint 的 m_c/n_c 约束

仅有 cache 尺度上界仍然不够，因为 A_c 和 B_c 的页覆盖会随 m_c 与 n_c 同步放大。若页大小为 P ，则两者的近似页覆盖为

$$\frac{m_c k_c S_{\text{data}}}{P}, \quad \frac{k_c n_c S_{\text{data}}}{P},$$

对应的近似 TLB 裁剪条件可写为

$$\frac{m_c k_c S_{\text{data}}}{P} \lesssim E_{A, \text{TLB}}, \quad \frac{k_c n_c S_{\text{data}}}{P} \lesssim E_{B, \text{TLB}}.$$

这些式子在本节中承担第二层 pruning 的作用。当前代码没有控制 huge page、STLB、NUMA 归属或线程绑定，因此这里仍按近似约束处理。对当前机器，取 $P = 4096$ 、L1 DTLB 条目数 72、L2 DTLB 条目数 3072，并采用保守的有效预算 $E_{A, \text{eff}} = 54$ 与 $E_{B, \text{eff}} = 192$ ；其中 192 用于第一轮单线程基础候选生成时抑制过宽 B_c 。则

$$\text{pages}(A_c) \approx \frac{m_c k_c}{1024}, \quad \text{pages}(B_c) \approx \frac{k_c n_c}{1024}.$$

于是, 本节使用的 TLB-aware pruning 规则为: 若某个 m_c 候选使 $\text{pages}(A_c) > 54$, 则将其从 $\mathcal{M}_{\text{cache}}$ 中剔除; 若某个 n_c 候选使 $\text{pages}(B_c) > 192$, 则将其从 $\mathcal{N}_{\text{cache}}$ 中剔除。这里的 $E_{B,\text{eff}}$ 小于硬件 L2 DTLB 容量, 用于在第一轮基础搜索中先裁掉过宽的 B_c 面板。

B.3.3 综合 cache 与 TLB 约束的单线程基础候选生成

对给定的 (m_r, n_r, k_c) 候选, 首先由 cache 上界生成原始离散集合

$$\mathcal{M}_{\text{cache}}(m_r, n_r, k_c) = \text{Align}_{g_m}^\downarrow(\{\beta m_c^{\max}(k_c) : \beta \in \mathcal{A}_m\}),$$

$$\mathcal{N}_{\text{cache}}(m_r, n_r, k_c) = \text{Align}_{g_n}^\downarrow(\{\gamma n_c^{\max}(k_c) : \gamma \in \mathcal{A}_n\}),$$

再由 TLB / page-footprint 做一次裁剪, 得到

$$\mathcal{M}_{\text{pruned}} \subseteq \mathcal{M}_{\text{cache}}, \quad \mathcal{N}_{\text{pruned}} \subseteq \mathcal{N}_{\text{cache}}.$$

为控制第一轮单线程基础搜索规模, 这里取 $g_m = 32$ 、 $g_n = 64$, 并令 $\mathcal{A}_m = \mathcal{A}_n = \{0.5, 0.75, 1.0\}$ 。这里的 $\text{Align}^\downarrow$ 表示向下对齐到不超过目标值的最近粒度倍数。按同一规则, 可对第 B.2 节保留下来的全部 (m_r, n_r, k_c) 组合逐项构造单线程基础候选, 如表 10 所示。

(m_r, n_r, k_c)	$m_c^{\max}$	$\mathcal{M}_{\text{cache}}$	$\mathcal{M}_{\text{pruned}}$	$n_c^{\max}$	$\mathcal{N}_{\text{cache}}$	$\mathcal{N}_{\text{pruned}}$
(8, 8, 288)	227.6	{96, 160, 224}	{96, 160}	910.2	{448, 640, 896}	{448, 640}
(8, 8, 384)	170.7	{64, 128, 160}	{64, 128}	682.7	{320, 512, 640}	{320, 512}
(12, 8, 256)	256.0	{128, 192, 256}	{128, 192}	1024.0	{512, 768, 1024}	{512, 768}
(12, 8, 352)	186.2	{64, 128, 160}	{64, 128}	744.7	{320, 512, 704}	{320, 512}
(12, 8, 416)	157.5	{64, 96, 128}	{64, 96, 128}	630.2	{256, 448, 576}	{256, 448}
(13, 8, 224)	292.6	{128, 192, 288}	{128, 192}	1170.3	{576, 832, 1152}	{576, 832}
(13, 8, 320)	204.8	{96, 128, 192}	{96, 128}	819.2	{384, 576, 768}	{384, 576}
(13, 8, 384)	170.7	{64, 128, 160}	{64, 128}	682.7	{320, 512, 640}	{320, 512}
(4, 16, 192)	341.3	{160, 256, 320}	{160, 256}	1365.3	{640, 1024, 1344}	{640, 1024}
(4, 16, 256)	256.0	{128, 192, 256}	{128, 192}	1024.0	{512, 768, 1024}	{512, 768}
(4, 16, 320)	204.8	{96, 128, 192}	{96, 128}	819.2	{384, 576, 768}	{384, 576}
(5, 16, 160)	409.6	{192, 288, 384}	{192, 288}	1638.4	{768, 1216, 1600}	{768, 1216}
(5, 16, 192)	341.3	{160, 256, 320}	{160, 256}	1365.3	{640, 1024, 1344}	{640, 1024}
(5, 16, 256)	256.0	{128, 192, 256}	{128, 192}	1024.0	{512, 768, 1024}	{512, 768}
(6, 16, 128)	512.0	{256, 384, 512}	{256, 384}	2048.0	{1024, 1536, 2048}	{1024, 1536}
(6, 16, 160)	409.6	{192, 288, 384}	{192, 288}	1638.4	{768, 1216, 1600}	{768, 1216}
(6, 16, 224)	292.6	{128, 192, 288}	{128, 192}	1170.3	{576, 832, 1152}	{576, 832}

TABLE 10 – 第一轮单线程基础候选中的 M_c/N_c 裁剪结果

表 10 给出的 $\mathcal{M}_{\text{pruned}}$ 与 $\mathcal{N}_{\text{pruned}}$ 构成了单线程或弱并行情形下的基础候选集合。它们对应“cache 先给基础尺度, TLB 再裁掉偏大的点”这一两层结构。后续实验再讨论这些基础候选如何进一步收缩。

第一轮单线程 coarse screening 覆盖了全部保留的 (m_r, n_r, k_c) 组合; 在此基础上, 只对局部优胜的 5 组 `KernelShape/Kc` 继续做第二轮精筛。第二轮使用的更密 M_c/N_c 网格如表 11 所示。

内核	m_r	n_r	k_c	M_c 候选	N_c 候选	组合数
8x8	8	8	288	{96, 128, 160, 192}	{448, 512, 576, 640}	16
8x8	8	8	384	{64, 96, 128}	{320, 384, 448, 512}	12
4x16	4	16	192	{160, 192, 224, 256, 288}	{640, 768, 896, 1024}	20
4x16	4	16	256	{128, 160, 192}	{512, 576, 640, 704, 768}	15
4x16	4	16	320	{96, 128, 160}	{384, 448, 512, 576}	12

TABLE 11 – 第二轮精筛使用的 M_c/N_c 候选网格

因此，第二轮总候选数为 $16 + 12 + 20 + 15 + 12 = 75$ 。这些组合在第一轮已知优胜区域附近补足中间点，从而提高单线程 blocked-size 结论的稳定性。

F 线程感知 $kCacheBlockM/N_c$ 调优热图补充

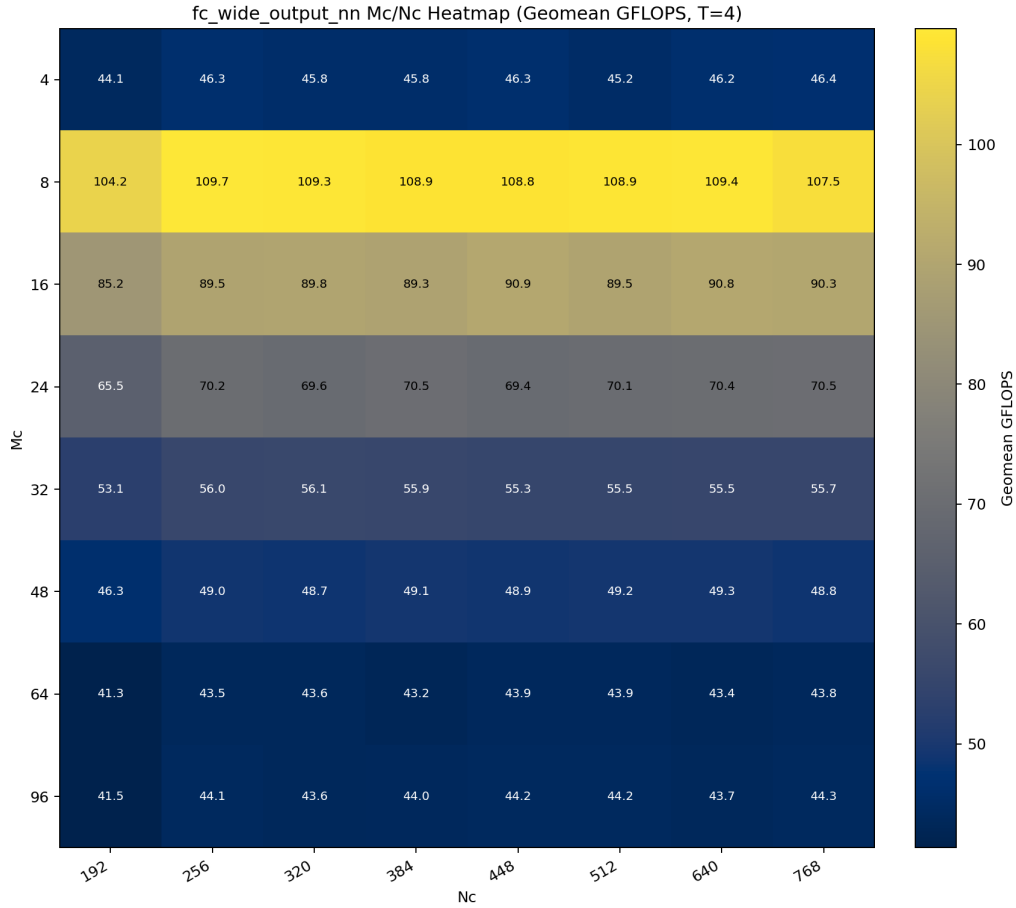


FIGURE 13 – 线程感知 $kCacheBlockM/N_c$ 调优的代表性热图 ($T = 4$, `fc_wide_output_nn`)。高性能区域主要集中在 $kCacheBlockM = 8$ (有效 $M_c = 64$) 一行，而 N_c 方向表现为相对平坦的平台。

G 固定方阵上的线程扩展补充实验

除前述项目 workload 外，本文还对 64×64 、 128×128 、 256×256 和 512×512 四组固定方阵做了多实现线程扩展补充实验；图 14 汇总了 blas、omp、omp_blocked、omp_blocked_packab、omp_gotoblas_avx2 以及 AVX-512 unroll+prefetch 路径在 1-8 个物理线程范围内的结果。

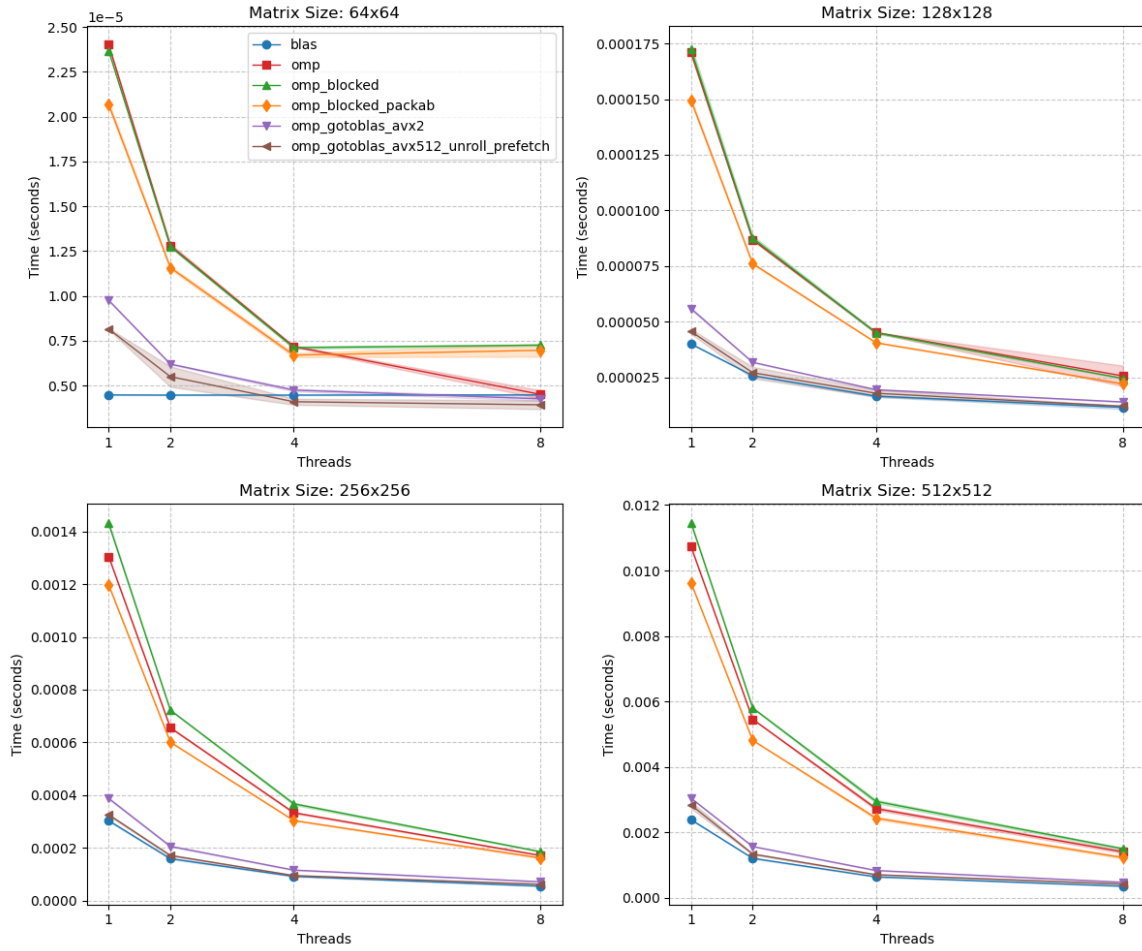


FIGURE 14 – 固定方阵上的多实现线程扩展补充实验。四个子图分别对应 64×64 、 128×128 、 256×256 与 512×512 ，纵轴为执行时间，横轴为线程数。

图 14 显示， 64×64 下 blas 基本保持近似单线程路径，线程数变化对其影响较小，而几条 OpenMP 路径通常要到 4-8 线程附近才接近这一水平；在 128×128 至 512×512 的区间里，omp_gotoblas_avx2 与 omp_gotoblas_avx512_unroll_prefetch 的时间曲线下降更快，其中后者在图中覆盖的固定方阵与 1-8 线程范围内整体上通常略优于 AVX2 路径。